

TUGAS AKHIR - IT184802

PENGEMBANGAN APLIKASI *SIGN LANGUAGE-TO-SPEECH* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BISINDO

MUHAMMAD ZULFIKAR FAUZI

NRP 05311840000012

Dosen Pembimbing I

Dr. Rizka Wakhidatus Sholikhah, S.Kom

NIP 1993202012054

Dosen Pembimbing II

Ridho Rahman Hariadi, S.Kom., M.Sc.

NIP 19870213 201404 1 001

DEPARTEMEN TEKNOLOGI INFORMASI

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2022

UNDERGRADUATE THESIS - IT184802

THE DEVELOPMENT OF A SIGN LANGUAGE-TO-SPEECH APPLICATION USING MACHINE LEARNING APPROACH FOR BISINDO

MUHAMMAD ZULFIKAR FAUZI

NRP. 05311840000012

Advisor I

Dr. Rizka Wakhidatus Sholikhah, S.Kom

NIP. 1993202012054

Advisor II

Ridho Rahman Hariadi, S.Kom., M.Sc.

NIP. 19870213 201404 1 001

INFORMATION TECHNOLOGY DEPARTMENT

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2022



TUGAS AKHIR - IT184802

PENGEMBANGAN APLIKASI *SIGN LANGUAGE-TO-SPEECH* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BISINDO

MUHAMMAD ZULFIKAR FAUZI

NRP. 05311840000012

Dosen Pembimbing I

Dr. Rizka Wakhidatus Sholikhah, S.Kom

NIP. 1993202012054

Dosen Pembimbing II

Ridho Rahman Hariadi, S.Kom., M.Sc.

NIP. 19870213 201404 1 001

DEPARTEMEN TEKNOLOGI INFORMASI

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2022



UNDERGRADUATE THESIS - IT184802

THE DEVELOPMENT OF A SIGN LANGUAGE-TO-SPEECH APPLICATION USING MACHINE LEARNING APPROACH FOR BISINDO

MUHAMMAD ZULFIKAR FAUZI

NRP. 05311840000012

Advisor I

Dr. Rizka Wakhidatus Sholikah, S.Kom

NIP. 1993202012054

Advisor II

Ridho Rahman Hariadi, S.Kom., M.Sc.

NIP. 19870213 201404 1 001

INFORMATION TECHNOLOGY DEPARTMENT

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2022

LEMBAR PENGESAHAN

PENGEMBANGAN APLIKASI *SIGN LANGUAGE-TO-SPEECH* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BISINDO

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
Memperoleh gelar Sarjana Komputer pada
Program Studi S-1 Departemen Teknologi Informasi
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh : **MUHAMMAD ZULFIKAR FAUZI**

NRP. 05311840000012

Disetujui oleh Tim Penguji Tugas Akhir:

Dr. Rizka Wakhidatus Sholikhah, S.Kom
NIP. 1993202012054

(Pembimbing 1)

Ridho Rahman Hariadi, S.Kom., M.Sc.
NIP. 19870213 201404 1 001

(Pembimbing 2)

Dr.tech.Ir. Raden Venantius Hari Ginardi, M.Sc
NIP. 196505181992031003

(Penguji 1)

Edwin Riksakomara, S.Kom., M.T.
NIP. 196907252003121001

(Penguji 2)

SURABAYA

Juli, 2022

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP : Muhammad Zulfikar Fauzi / 053118400000012

Departemen : Teknologi Informasi

Dosen Pembimbing I / NIP : Dr. Rizka Wakhidatus Sholikhah, S.Kom / 1993202012054

Dosen Pembimbing II / NIP : Ridho Rahman Hariadi, S.Kom., M.Sc. / 19870213 201404 1001

Dengan ini menyatakan bahwa Buku Tugas Akhir dengan judul “*PENGEMBANGAN APLIKASI SIGN LANGUAGE-TO-SPEECH MENGGUNAKAN PENDEKATAN MACHINE LEARNING PADA BISINDO*” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

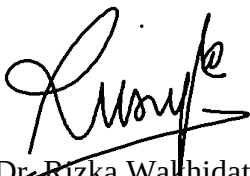
Surabaya, Juli 2022

Mengetahui,

Dosen Pembimbing I

Dosen Pembimbing II

Mahasiswa,



(Dr. Rizka Wakhidatus
Sholikhah, S. Kom)

NIP. 1993202012054



(Ridho Rahman Hariadi,
S.Kom., M.Sc.)

NIP. 19870213 201404 1001

(Muhammad Zulfikar
Fauzi)

NRP. 053118400000012

PENGEMBANGAN APLIKASI *SIGN LANGUAGE-TO-SPEECH* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BISINDO

Nama Mahasiswa / NRP : Muhammad Zulfikar Fauzi / 05311840000012
Departemen : Teknologi Informasi FTEIC – ITS
Dosen Pembimbing 1 : Dr. Rizka Wakhidatus Sholikhah, S. Kom
Dosen Pembimbing 2 : Ridho Rahman Hariadi, S.Kom., M.Sc.

Abstrak

Tuna rungu adalah seseorang yang mengalami gangguan dalam pendengarannya, gangguan ini dapat dialami baik dari lahir ataupun setelah mengalami peristiwa tertentu. Penyandang tuli memiliki cara tersendiri untuk berkomunikasi dengan orang lain yakni melalui bahasa isyarat.

BISINDO adalah salah satu bahasa isyarat yang ada di Indonesia. Bahasa isyarat ini muncul secara alami di dalam budaya Indonesia dan cenderung lebih sering dipakai dikarenakan dianggap lebih mudah bagi masyarakat Indonesia dibandingkan bahasa isyarat ASL maupun SIBI.

Tidak semua orang dapat mengartikan bahasa isyarat sehingga penyandang tuli terkadang mengalami kesulitan saat hendak berkomunikasi. Maka dari itu diperlukan suatu media yang dapat menjembatani komunikasi. Dengan menggunakan aplikasi yang dapat menerjemahkan dari bahasa isyarat dan dapat mengeluarkan suara, komunikasi antara penyandang tuli dan orang normal akan lebih mudah dilakukan.

Oleh karena itu pada penelitian ini dibangun sebuah aplikasi *sign language-to-speech* untuk menerjemahkan bahasa isyarat BISINDO ke dalam bentuk suara dengan pendekatan *machine learning*. *Output* berupa suara ini akan mempermudah bagi pengguna untuk berkomunikasi dengan lawan bicara dan akan mempermudah lawan bicara untuk memahami bahasa isyarat dan akan meningkatkan kualitas komunikasi.

Dengan menggunakan *dataset* penelitian ini menghasilkan akurasi *model* sebesar 98% dengan menggunakan metode Support Vector Machine. Tetapi tingkat akurasi menurun drastis mencapai 78% dalam uji coba yang dilakukan langsung pada pengguna. Hasil implementasi *Sign Language-to-Speech* berhasil mengeluarkan *audio* tanpa menggunakan koneksi internet.

Kata kunci: *Penyandang Tuli, BISINDO, Machine Learning, Speech*

THE DEVELOPMENT OF A SIGN LANGUAGE-TO-SPEECH APPLICATION USING MACHINE LEARNING APPROACH FOR BISINDO

Student Name / Registration : Muhammad Zulfikar Fauzi / 05311840000012
Number
Department : Information Technology ELECTICS – ITS
Advisor 1 : Dr. Rizka Wakhidatus Sholikhah, S. Kom
Advisor 2 : Ridho Rahman Hariadi, S.Kom., M.Sc.

Abstract

A deaf person is someone who has a hearing loss, this disorder can be experienced either from birth or after experiencing certain events. Deaf people have their own way of communicating with other people, namely through sign language.

BISINDO is one of the sign languages in the world. This sign language appears naturally in Indonesian culture and tends to be used more often because it is considered easier for Indonesian people than other sign languages.

Not everyone can interpret sign language, so deaf people sometimes have difficulty communicating. Therefore we need a media that can bridge communication. By using an application that can translate from sign language and can produce sounds, communication between deaf people and normal people will be easier to do.

Therefore, in this study, a sign language-to-speech application was built to translate BISINDO sign language into voice using a machine learning approach. The output in the form of sound will make it easier for users to communicate with the other person and will make it easier for the other person to understand sign language and will improve the quality of communication.

By using this research dataset, the model accuracy is 98% using the Support Vector Machine method. But the accuracy rate dropped dramatically to 78% in tests conducted directly on users. The results of the implementation of Sign Language-to-Speech successfully output audio without using an internet connection.

Keywords: Deaf, BISINDO, Machine Learning, Speech

KATA PENGANTAR

Puji syukur penulis ucapkan kepada Tuhan Yang Maha Esa atas pimpinan, penyertaan, dan karunia-Nya sehingga penulis dapat menyelesaikan Tugas Akhir yang berjudul :

PENGEMBANGAN APLIKASI *SIGN LANGUAGE-TO-SPEECH* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BISINDO

Pengerjaan tugas akhir ini dilakukan untuk memenuhi salah satu syarat meraih gelar Sarjana di Departemen Teknologi Informasi Fakultas Teknologi Elektro dan Informatika Cerdas Institut Teknologi Sepuluh Nopember. Dengan selesainya tugas akhir ini diharapkan apa yang telah dikerjakan penulis dapat memberikan manfaat bagi perkembangan ilmu pengetahuan terutama di bidang teknologi informasi serta bagi diri penulis sendiri selaku peneliti.

Penulis ingin menyampaikan rasa terima kasih kepada banyak pihak yang telah membimbing, menemani dan membantu penulis selama masa pengerjaan tugas akhir maupun masa studi.

1. Allah SWT atas segala nikmat, rahmat, dan petunjuk yang telah diberikan kepada penulis.
2. Kedua orang tua, adik - adik penulis yang senantiasa mendukung, memberi semangat dan mendoakan sehingga penulis dapat menyelesaikan Tugas Akhir dengan baik.
3. Seluruh keluarga penulis yang senantiasa mendukung, memberi semangat dan mendoakan sehingga penulis dapat menyelesaikan Tugas Akhir dengan baik.
4. Bu Rizka Wakhidatus Sholikhah dan pak Ridho Rahman Hariadi selaku dosen pembimbing I dan II penulis dan mentor yang telah memberikan ilmu, pengalaman, arahan, serta bantuan-bantuan lainnya selama penyusunan tugas akhir.
5. Seluruh teman departemen Teknologi Informasi yang membantu menyemangati dan memberikan ide – ide selama masa studi.

Penulis memohon maaf apabila dalam tugas akhir ini masih banyak kekurangan. Penulis mengharapkan kritik dan saran yang bersifat membangun untuk penyempurnaan tugas akhir ini.

Surabaya, 18 Maret 2022

Muhammad Zulfikar Fauzi

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERNYATAAN ORISINALITAS	i
Abstrak.....	ii
Abstract.....	iii
KATA PENGANTAR.....	iv
DAFTAR ISI	v
DAFTAR GAMBAR.....	viii
DAFTAR TABEL	x
DAFTAR KODE SUMBER.....	xi
1. BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Batasan Masalah.....	2
1.4 Tujuan Penelitian.....	2
1.5 Manfaat Penelitian.....	2
2. BAB II TINJAUAN PUSTAKA	3
2.1 Tinjauan Penelitian Sebelumnya	3
2.1.1 Jurnal Terkait Pendekatan <i>Machine Learning</i> Untuk <i>Sign Language Recognition</i> 3	
2.1.2 Jurnal Terkait Penggunaan MediaPipe Untuk <i>Sign Language Recognition</i>	3
2.1.3 Jurnal Pengembangan MediaPipe.....	3
2.2 Dasar Teori.....	4
2.2.1 Penyandang Disabilitas.....	4
2.2.2 Bahasa Isyarat BISINDO.....	4
2.2.3 Algoritma Klasifikasi <i>Machine Learning</i> Random Forest	6
2.2.4 Algoritma Klasifikasi <i>Machine Learning</i> Support Vector Machine	6
2.2.5 Algoritma Klasifikasi <i>Machine Learning</i> K-Nearest Neighbor	7
2.2.6 Scikit-Learn	8
2.2.7 MediaPipe	8
2.2.8 Webcam	9
2.2.9 <i>Text-To-Speech</i>	9
2.2.10 OpenCV	9
3. BAB III METODOLOGI.....	10

3.1 Metode yang digunakan	10
3.1.1 Metode Pengambilan Data.....	10
3.1.2 Metode Pengolahan Data.....	11
3.1.3 Metode Pembuatan dan Evaluasi Model	13
3.1.4 Metode Pembuatan Aplikasi.....	15
3.2 Bahan dan Peralatan yang Digunakan.....	20
3.2.1 Laptop	20
3.2.2 <i>Dataset</i> Bahasa Isyarat BISINDO	20
3.2.3 <i>Software</i> Pendukung	21
3.3 Urutan Pelaksanaan Penelitian	21
3.3.1 Penyusunan Proposal Tugas Akhir	21
3.3.2 Studi Literatur dan Pengumpulan Data.....	21
3.3.3 Desain Sistem dan Implementasi.....	21
3.3.4 Pengujian dan Evaluasi.....	21
3.3.5 Penyusunan Laporan Tugas Akhir.....	22
4. BAB IV HASIL DAN PEMBAHASAN	23
4.1 Implementasi	23
4.1.1 <i>Dataset</i> Yang Digunakan.....	23
4.1.2 Instalasi dan Persiapan Fitur <i>Speech</i>	24
4.1.3 Implementasi Fitur Aplikasi	27
4.1.3.1 Deteksi BISINDO	28
4.1.3.2 Penyusunan Kalimat dan Fitur <i>Delete</i>	29
4.1.3.3 Fitur <i>Text-to-Speech</i>	31
4.2 Hasil	31
4.2.1 <i>Hyperparameter Tuning</i>	32
4.2.2 <i>Machine Learning Model</i>	33
4.2.3 Evaluasi Kerja Aplikasi	33
4.2.4 Fitur <i>Text-to-Speech</i>	51
4.3 Analisis.....	51
4.3.1 Analisis Performa Aplikasi.....	51
4.3.2 Analisis <i>Model</i>	52
4.3.3 Analisis Aplikasi.....	53
5. BAB V KESIMPULAN DAN SARAN	55
5.1 KESIMPULAN	55

5.2 SARAN	55
DAFTAR PUSTAKA.....	56
BIODATA PENULIS	58

DAFTAR GAMBAR

Gambar 2.1 Abjad dalam BISINDO (Klobility, 2019)	5
Gambar 2.2 Abjad dalam SIBI (Klobility, 2019)	5
Gambar 2.3 Algoritma Random Forest (JavaTpoint, 2011)	6
Gambar 2.4 Algoritma Support Vector Machine (JavaTpoint, 2011)	7
Gambar 3.1 Diagram Metode Penelitian	10
Gambar 3.2 Pengambilan Data Melalui Webcam	11
Gambar 3.3 Diagram Pembuatan <i>Dataset</i>	11
Gambar 3.4 <i>Pose Landmarks MediaPipe</i> (Google LLC, 2020)	12
Gambar 3.5 <i>Hand Landmarks MediaPipe</i> (Google LLC, 2020)	12
Gambar 3.6 Diagram Pembuatan Model	15
Gambar 3.7 Diagram Aplikasi	16
Gambar 3.8 Deteksi BISINDO	17
Gambar 3.9 Algoritma Pengurangan <i>False Positive Detection</i>	18
Gambar 3.10 Penyusunan Teks	19
Gambar 3.11 Algoritma <i>Text-to-Speech</i>	20
Gambar 4.1 Hasil <i>Running</i> Kode Pengecekan <i>Missing Class</i>	24
Gambar 4.2 <i>Add Voices</i>	25
Gambar 4.3 <i>Voice Packages</i>	25
Gambar 4.4 <i>Registry</i> Sebelum <i>Fix</i>	27
Gambar 4.5 <i>Registry</i> Sesudah <i>Fix</i>	27
Gambar 4.6 Pengujian 1 Nama Pengguna 1	34
Gambar 4.7 Pengujian 1 "AKU SEMANGAT" Pengguna 1	35
Gambar 4.8 Pengujian 1 Transportasi Pengguna 1	35
Gambar 4.9 Pengujian 1 Kata Ganti Keluarga Pengguna 1	36
Gambar 4.10 Pengujian 1 Nama Pengguna 2	37
Gambar 4.11 Pengujian 1 "AKU SEMANGAT" Pengguna 2	38
Gambar 4.12 Pengujian 1 Transportasi Pengguna 2	38
Gambar 4.13 Pengujian 1 Kata Ganti Keluarga Pengguna 2	39
Gambar 4.14 Pengujian 1 Nama Pengguna 3	40
Gambar 4.15 Pengujian 1 "AKU SEMANGAT" Pengguna 3	41
Gambar 4.16 Pengujian 1 Transportasi Pengguna 3	41
Gambar 4.17 Pengujian 1 Kata Ganti Keluarga Pengguna 3	42
Gambar 4.18 Pengujian 2 "PESAWAT, BUS, MOTOR"	43
Gambar 4.19 Pengujian 2 "IBU, TANTE, OM"	43
Gambar 4.20 Pengujian 3 "PESAWAT, BUS, MOTOR"	44
Gambar 4.21 Pengujian 3 "IBU, TANTE, OM"	44
Gambar 4.22 Pengujian 4 "PESAWAT, BUS, MOTOR"	45
Gambar 4.23 Pengujian 4 "IBU, TANTE, OM"	46
Gambar 4.24 Pengujian 5 "PESAWAT, BUS, MOTOR"	47
Gambar 4.25 Pengujian 6 "PESAWAT, BUS, MOTOR"	48
Gambar 4.26 Pengujian 6 "IBU, TANTE, OM"	48
Gambar 4.27 Pengujian 7 Pengguna Berada di Kanan dan Aplikasi gagal Memprediksi	49
Gambar 4.28 Pengujian 7 Pengguna berada di Kanan dan Aplikasi Berhasil Memprediksi ...	49
Gambar 4.29 Pengujian 7 Pengguna Berada di Kiri dan Aplikasi Gagal Memprediksi	50

Gambar 4.30 <i>Frame per second</i> menggunakan MediaPipe	52
Gambar 4.31 <i>Frame per second</i> tanpa MediaPipe	52

DAFTAR TABEL

Tabel 4.1 <i>Preview Dataset BISINDO</i>	23
Tabel 4.2 Hasil <i>Hyperparameter Tuning</i> Random Forest	32
Tabel 4.3 Hasil <i>Hyperparameter Tuning</i> SVM	32
Tabel 4.4 Hasil <i>Hyperparameter Tuning</i> SVM kernel poly	32
Tabel 4.5 Hasil <i>Hyperparameter Tuning</i> KNN	33
Tabel 4.6 Hasil <i>Testing Machine Learning Model</i>	33
Tabel 4.7 Hasil Pengujian Aplikasi pada Pengguna	50

DAFTAR KODE SUMBER

Kode Sumber 4.1 Kode Pengecekan <i>Missing Class</i>	24
Kode Sumber 4.2 <i>registry voice pack</i> Bahasa Indonesia.....	26
Kode Sumber 4.3 <i>Preprocessing Image Format</i>	28
Kode Sumber 4.4 <i>Feature Selection</i> dan Persiapan Variabel Untuk Deteksi	29
Kode Sumber 4.5 Kode Prediksi BISINDO	29
Kode Sumber 4.6 Pengecekan <i>Threshold</i> dan <i>Class Dupe</i>	29
Kode Sumber 4.7 Fitur <i>Delete</i>	30
Kode Sumber 4.8 Pengecekan Tipe <i>Class</i>	30
Kode Sumber 4.9 Kode Penyusunan Kalimat pada <i>Array</i> Teks.....	30
Kode Sumber 4.10 Kode Konfigurasi TTS	31
Kode Sumber 4.11 Kode Aktivasi TTS	31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Komunikasi merupakan hal yang selalu dilakukan oleh orang setiap harinya agar dapat mencapai suatu tujuan ataupun menambah relasi sosial akan tetapi tidak semuanya dapat melakukan komunikasi dengan normal selayaknya orang pada umumnya, penyandang disabilitas tuna rungu dan wicara adalah salah satu disabilitas yang menghadapi permasalahan saat berinteraksi sosial. Sebagian besar dari mereka menggunakan bahasa isyarat untuk berkomunikasi dengan lainnya.

Bahasa isyarat di Indonesia sendiri terbagi menjadi 2 yang sering digunakan yaitu SIBI (Sistem Isyarat Bahasa Indonesia) dan BISINDO (Bahasa Isyarat Indonesia). Perbedaan dari keduanya dapat dilihat dari penggunaan tangan, SIBI menggunakan satu tangan sedangkan BISINDO menggunakan kedua tangan. SIBI diadopsi sebagian besar dari bahasa isyarat Amerika. SIBI dianggap sulit bagi sebagian besar masyarakat Indonesia dikarenakan tingkat kompleksitasnya yang tinggi. BISINDO adalah bahasa isyarat yang muncul secara alami dalam budaya Indonesia, dibentuk oleh kelompok tuli Indonesia, dan digunakan dalam kehidupan sehari – hari. (Tempo Media Group, 2021)

Dalam data yang tercatat di sistem informasi manajemen penyandang disabilitas milik Kemensos terdapat 13.807 penyandang tunarungu dan 5.585 penyandang tunawicara pada saat penulisan tugas akhir ini. (Kementerian Sosial RI, 2018)

Tidak semua orang dapat mengartikan bahasa isyarat sehingga penyandang tuli terkadang mengalami kesulitan saat hendak berkomunikasi. Hal ini tentunya mempersulit penyandang tuli dalam mencapai tujuan tertentu bahkan bisa menutup beberapa prospek masa depan. Maka dari itu diperlukan suatu media yang dapat menjembatani komunikasi. Dengan menggunakan aplikasi yang dapat menerjemahkan dari bahasa isyarat dan dapat mengeluarkan suara, kendala yang dialami penyandang tuli dapat diminimalisir.

Oleh karena itu pada penelitian ini akan dikembangkan teknologi berbasis *machine learning* untuk mengartikan bahasa isyarat BISINDO ini. Akan digunakan *framework* MediaPipe sebagai solusi *machine learning* dalam penelitian ini. MediaPipe telah digunakan untuk mendeteksi bahasa isyarat menggunakan *computer vision* dengan tingkat akurasi tinggi berkisaran antara 86% hingga 91%. (Gomase, Dhanawade, Gurav, & Lokare, 2022)

1.2 Rumusan Masalah

Rumusan masalah pada pembuatan Tugas Akhir ini adalah sebagai berikut:

1. Bagaimana sistem dapat mendeteksi dan mengartikan bahasa isyarat BISINDO?
2. Bagaimana cara menghasilkan *output* berupa suara dari bahasa isyarat yang dilakukan oleh pengguna?

1.3 Batasan Masalah

Berdasarkan pada rumusan masalah, adapun batasan pada Tugas Akhir ini adalah sebagai berikut:

1. Menggunakan *framework machine learning* MediaPipe untuk mendeteksi dan ekstraksi titik *landmarks* tubuh.
2. Metode deteksi berbasis *computer vision* dan tidak menggunakan alat bantu eksternal.
3. Menggunakan metode *machine learning* Support Vector Machine.
4. Tidak menggunakan *Natural Language Processing*.
5. *Webcam* yang digunakan selama proses penelitian adalah *webcam* internal laptop.
6. Media *output* suara yang digunakan adalah *speaker* internal laptop.
7. Tidak mengimplementasikan sistem deteksi ekspresi muka dan gerakan mulut.
8. Aplikasi dikembangkan pada sistem operasi Windows 10.
9. Narator *text-to-speech* menggunakan Microsoft *Speech API* (SAPI).

1.4 Tujuan Penelitian

Tujuan pembuatan Tugas Akhir ini adalah sebagai berikut.

1. Mengembangkan sistem yang dapat mendeteksi dan mengartikan bahasa isyarat BISINDO.
2. Mengembangkan sistem yang dapat mengeluarkan *output* berupa suara dari bahasa isyarat yang dilakukan oleh pengguna.

1.5 Manfaat Penelitian

Tugas akhir ini diharapkan dapat memiliki beberapa manfaat sebagai berikut.

1. Membantu pengguna dalam menerjemahkan bahasa isyarat kepada lawan bicaranya.
2. Meningkatkan mutu interaksi penyandang tuli dengan khalayak umum.
3. Membuka peluang baru bagi penyandang tuli dalam berbagai prospek di masa mendatang.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Penelitian Sebelumnya

Penelitian ini memiliki beberapa tinjauan penelitian sebelumnya untuk dijadikan bahan referensi pada masa penelitian.

2.1.1 Jurnal Terkait Pendekatan *Machine Learning* Untuk *Sign Language Recognition*

Penelitian terdahulu terkait *sign language recognition* telah dilakukan dengan menggunakan pendekatan beberapa metode *machine learning* (Halder & Toyade, 2021). Penelitian ini dilakukan karena penelitian *Sign Language Recognition* berbasis sensor dirasa tidak nyaman serta memakan biaya tinggi. Maka dari itu penelitian ini mengatasi kelemahan tersebut dengan memanfaatkan proyek MediaPipe untuk melakukan ekstraksi *landmark* dari gambar tangan serta memanfaatkan beberapa metode *machine learning* untuk dibandingkan hasilnya. Beberapa bahasa isyarat mulai dari ASL, India, Italia, serta Turkey dijadikan sebagai *dataset*.

Penelitian ini terdiri dari beberapa tahapan yaitu melakukan *pre-process* sebuah gambar untuk mendapatkan *multi-hand landmark* dengan menggunakan MediaPipe. Setelah itu, melakukan pembersihan data serta normalisasi lalu melakukan prediksi menggunakan algoritma *machine learning*. Penelitian ini mendapatkan hasil akurasi klasifikasi tertinggi pada metode SVM di semua bahasa isyarat dengan rentang nilai akurasi terendah sebesar 96.22% pada bahasa Turkey hingga nilai akurasi tertinggi 99.29% pada bahasa India.

2.1.2 Jurnal Terkait Penggunaan MediaPipe Untuk *Sign Language Recognition*

Penelitian terdahulu terkait *sign language recognition* telah dilakukan dengan menggunakan pendekatan *deep learning* (Gomase, Dhanawade, Gurav, & Lokare, 2022). Penelitian ini mengimplementasikan MediaPipe dalam *sign language recognition* untuk bahasa isyarat *American Sign Language*, bahasa isyarat yang dideteksi berupa huruf A hingga Z. Dalam penelitian ini yang digunakan adalah solusi *hand* dari MediaPipe.

Tingkat akurasi penggunaan mediapipe dengan spesifikasi 17 FPS dalam sistem yang dibangun dalam jurnal ini adalah berkisaran dari 86% hingga 91%. Untuk MediaPipenya sendiri dapat mendeteksi telapak tangan dengan rata – rata akurasi sebesar 95.7%.

2.1.3 Jurnal Pengembangan MediaPipe

Penelitian terdahulu terkait riset dan pengembangan MediaPipe (Zhang, et al., 2020). Menurut jurnal tersebut MediaPipe adalah solusi *real-time on-device hand tracking* yang memprediksi tulang tangan manusia dari kamera berformat RGB. *Hand tracking* adalah komponen vital untuk menyediakan cara yang alami untuk berinteraksi didalam *Augmented Reality* (AR) ataupun *Virtual Reality* (VR) dan sudah menjadi topik hangat dalam industri berbasis *computer vision*.

2.2 Dasar Teori

Pada penelitian ini terdapat berbagai dasar teori yang perlu dipahami terlebih dahulu guna mempermudah pengerjaan selama masa penelitian.

2.2.1 Penyandang Disabilitas

Pemerintah Indonesia telah menetapkan pengertian resmi tentang siapa yang dimaksud penyandang disabilitas di Indonesia dan pengkategorianannya. UU Nomor 8 tahun 2016 Pasal 1 ayat 1 mendefinisikan penyandang disabilitas sebagai:

Setiap orang yang mengalami keterbatasan fisik, intelektual, mental, dan/atau sensorik dalam jangka waktu yang lama yang dalam berinteraksi dengan lingkungan dapat mengalami hambatan dan kesulitan untuk berpartisipasi secara penuh dan efektif dengan warga negara lainnya berdasarkan kesamaan hak. (Widinarsih, 2019)

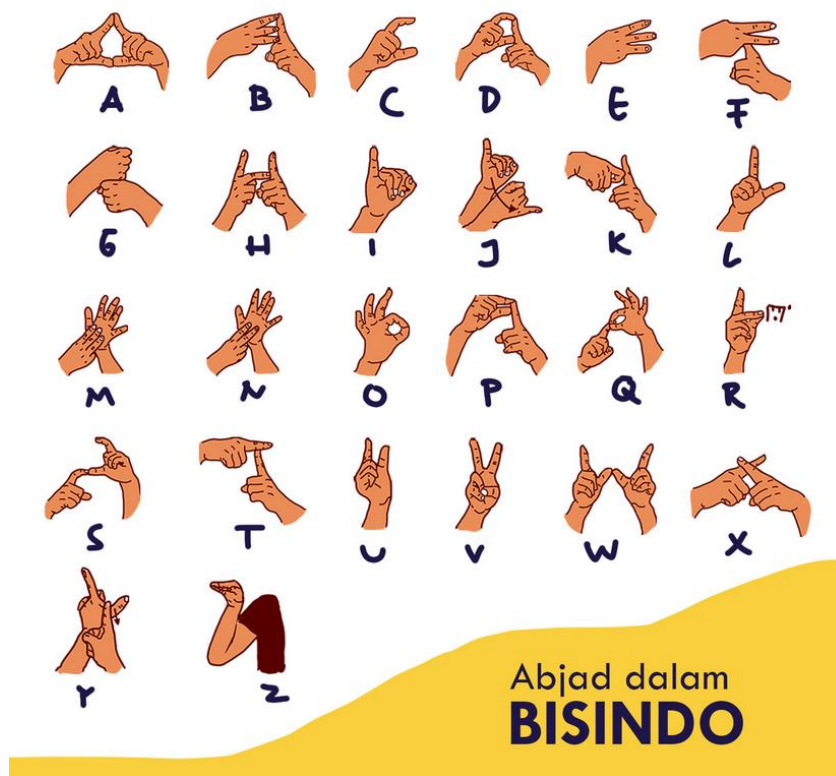
2.2.2 Bahasa Isyarat BISINDO

BISINDO (Bahasa Isyarat Indonesia) adalah bahasa isyarat yang muncul secara alami dalam budaya Indonesia dan digunakan dalam kehidupan sehari-hari. BISINDO memiliki beberapa variasi di tiap daerah. Bahasa isyarat ini mempunyai tata bahasa yang berbeda dengan bahasa lisan yang digunakan orang-orang mendengar pada umumnya, mencakup semua unsur, mulai dari fonologi, morfologi, sintaksis, pragmatis, dan unsur lainnya. Perbedaan SIBI dan BISINDO yang cukup terlihat adalah gerakan tangannya. SIBI (Sistem Isyarat Bahasa Indonesia) menggunakan satu tangan untuk mengisyaratkan abjad, sedangkan BISINDO membutuhkan dua tangan. (Tempo Media Group, 2021)

Keputusan dari Kementerian Pendidikan dan Budaya Nomor 0190/P/1994 tanggal 1 Agustus 1994 untuk membuat Kamus Sistem Isyarat Bahasa Indonesia (SIBI) menimbulkan perselisihan antara Komunitas Tuli, yaitu termasuk Gerakan untuk Kesejahteraan Tuna Rungu Indonesia (Gerkatin), dengan tim perumus Kamus SIBI. Dewan Pengurus Pusat Gerkatin mengangkat isu ini untuk didiskusikan bersama di Kongres Nasional keenam Gerkatin di Bali pada tahun 2002. Berdasarkan hasil keputusan dari Kongres Nasional tersebut, nama Bahasa Isyarat Indonesia ditetapkan secara resmi dan kemudian disingkat menjadi Bisindo dengan tujuan mempertahankan bahasa isyarat alamiah. (PUSBISINDO, 2022)

Kemudian untuk memperkuat status bahasa isyarat sebagai bahasa, Gerkatin mulai bekerjasama dengan The Center for Sign Linguistics and Deaf Studies The Chinese University of Hong Kong dan Fakultas Ilmu Pengetahuan Budaya Universitas Indonesia. Para perwakilan Tuli di setiap daerah di seluruh Indonesia juga meminta agar segera mendirikan wadah pengembangan dan penelitian bahasa isyarat di Kongres Nasional Gerkatin di Makassar pada tahun 2006. Pada akhirnya, Pusat Bahasa Isyarat Indonesia (Pusbisindo) diresmikan dalam Rapat Kerja Nasional Pertama pada tahun 2009 di Jakarta. (PUSBISINDO, 2022)

Dapat dilihat dari Gambar 2.1 dan Gambar 2.2 perbedaan pengisyratan pada bahasa isyarat BISINDO dan SIBI. BISINDO cenderung memakai kedua tangan sedangkan SIBI hanya menggunakan 1 tangan pada pengisyratan huruf abjad.



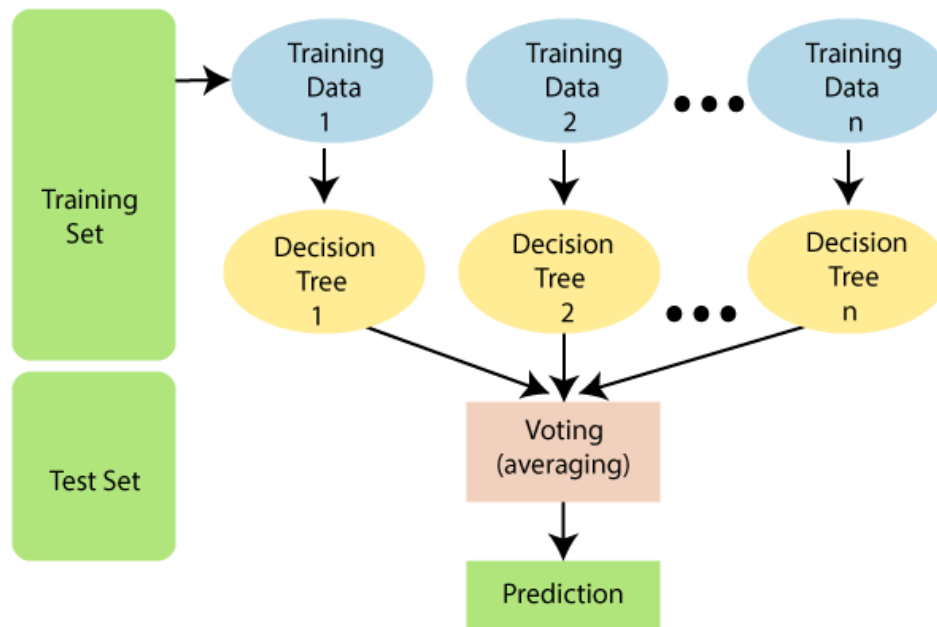
Gambar 2.1 Abjad dalam BISINDO (Klobility, 2019)



Gambar 2.2 Abjad dalam SIBI (Klobility, 2019)

2.2.3 Algoritma Klasifikasi *Machine Learning* Random Forest

Random Forest adalah algoritma *supervised machine learning* populer yang dapat digunakan untuk masalah klasifikasi dan regresi. Hal ini didasarkan pada konsep *ensemble learning*, yang merupakan proses menggabungkan beberapa *classifier* untuk memecahkan masalah yang kompleks dan untuk meningkatkan kinerja *model*.



Gambar 2.3 Algoritma Random Forest (JavaTpoint, 2011)

Random Forest adalah *classifier* yang berisi sejumlah *decision tree* pada berbagai *subset* dari *dataset* yang diberikan dan mengambil rata-rata untuk meningkatkan akurasi prediksi dari *dataset* tersebut. Random Forest mengambil prediksi dari setiap *tree* dan berdasarkan suara terbanyak dari prediksi, dan memprediksi hasil akhir.

Beberapa kelebihan menggunakan algoritma ini adalah sebagai berikut:

- Membutuhkan waktu training yang lebih cepat dibanding algoritma lainnya.
- Memiliki akurasi yang tinggi, berjalan secara efisien untuk dataset skala besar.
- Akurasi terjaga meskipun sebagian besar data menghilang. (JavaTpoint, 2011)
- Tidak memerlukan normalisasi data.

Beberapa kelemahan menggunakan algoritma ini adalah sebagai berikut:

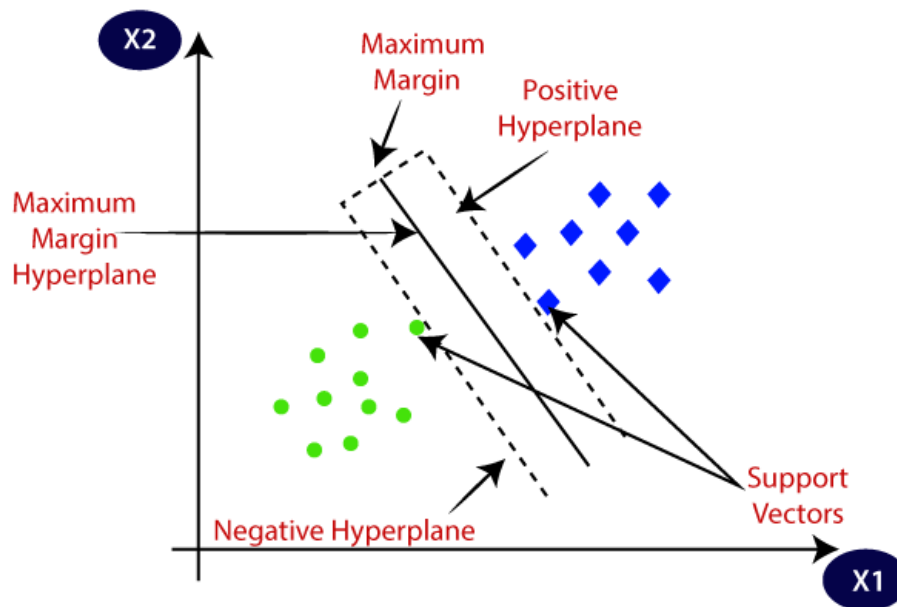
- Kompleksitas parameter yang tinggi sehingga diperlukan waktu lebih untuk dilakukan optimalisasi dan *hypertuning* parameter.
- Bias terhadap variabel dengan *level* yang lebih banyak.

2.2.4 Algoritma Klasifikasi *Machine Learning* Support Vector Machine

Support vector machine berdasarkan teori pembelajaran statistik oleh Vapnik et al mengusulkan metode pembelajaran baru, yang dibangun berdasarkan jumlah sampel yang

terbatas dalam informasi yang terkandung dalam teks pelatihan yang ada untuk mendapatkan hasil klasifikasi terbaik (Zhang Y. , 2012).

SVM bekerja dengan memetakan data ke ruang fitur berdimensi tinggi sehingga titik data dapat dikategorikan, bahkan ketika data tidak dapat dipisahkan secara linier. Pemisah antar kategori ditemukan, kemudian data ditransformasikan sedemikian rupa sehingga pemisah tersebut dapat digambarkan sebagai *hyperplane*. Dapat dilihat pada Gambar 2.4 untuk *hyperplane* yang digunakan pada metode Support Vector Machine.



Gambar 2.4 Algoritma Support Vector Machine (JavaTpoint, 2011)

Kelebihan dari menggunakan algoritma Support Vector Machine adalah sebagai berikut:

- Bekerja sangat baik dengan margin pemisahan yang jelas.
- Efektif dalam ruang dimensi tinggi.
- Efektif dalam kasus di mana jumlah dimensi lebih besar dari jumlah sampel.
- Menggunakan subset titik pelatihan dalam fungsi keputusan (disebut vektor pendukung), sehingga juga hemat memori.

Kekurangan dari menggunakan algoritma Support Vector Machine adalah sebagai berikut:

- Tidak bekerja dengan baik ketika *dataset* berjumlah besar karena waktu pelatihan yang dibutuhkan lebih tinggi.
- Tidak bekerja dengan baik ketika *dataset* memiliki lebih banyak *noise*.
- SVM tidak secara langsung memberikan perkiraan probabilitas.

2.2.5 Algoritma Klasifikasi *Machine Learning* K-Nearest Neighbor

Algoritma k-nearest neighbor, juga dikenal sebagai KNN atau k-NN, adalah *supervised learning classifier* non-parametrik, yang menggunakan kedekatan untuk membuat klasifikasi

atau prediksi tentang pengelompokan titik data individual. Meskipun dapat digunakan untuk masalah regresi atau klasifikasi umumnya digunakan dalam klasifikasi, bekerja dengan asumsi bahwa titik serupa dapat ditemukan di dekat satu dengan yang lain.

Untuk masalah klasifikasi, label kelas ditetapkan berdasarkan suara mayoritas, label yang paling sering muncul di sekitar titik data tertentu digunakan untuk melakukan klasifikasi. Nilai k pada algoritma k -NN mendefinisikan berapa banyak tetangga yang akan diperiksa untuk menentukan klasifikasi titik *query* tertentu. Misalnya, jika $k=1$, *instance* akan diklasifikasi ke kelas yang sama dengan tetangga terdekatnya. (IBM, 1986)

Beberapa kelebihan menggunakan algoritma ini adalah sebagai berikut:

- Mudah diimplementasikan.
- Mudah beradaptasi terhadap data baru yang ditambahkan kedalam sampel training.
- Parameter yang digunakan sedikit sehingga mudah dilakukan *tuning*.

Beberapa kelemahan menggunakan algoritma ini adalah sebagai berikut:

- Memiliki performa yang kurang terhadap data dengan dimensi tinggi.
- Rentan terjadi overfitting data.
- Memerlukan *memory* dan *data storage* yang lebih tinggi dibandingkan metode machine learning lainnya.

2.2.6 Scikit-Learn

Scikit-learn adalah modul Python yang mengintegrasikan berbagai algoritma *machine learning* canggih untuk masalah *supervised* dan *unsupervised* skala menengah. Paket ini berfokus pada *machine learning* ke non-spesialis menggunakan bahasa tingkat tinggi umum. Penekanan diletakkan pada kemudahan penggunaan, kinerja, dokumentasi, dan konsistensi API. Ini memiliki ketergantungan minimal dan didistribusikan di bawah lisensi BSD yang disederhanakan, mendorong penggunaannya dalam pengaturan akademik dan komersial. (Pedregosa, et al., 2011)

2.2.7 MediaPipe

MediaPipe adalah *framework open-source* yang dikembangkan oleh Google untuk membangun multimodal, *cross platform*, *Machine Learning* Pipelines. MediaPipe adalah solusi *machine learning* untuk berbagai macam deteksi pada *computer vision*, antara lain *hand gesture*, *face detection*, *pose*, *holistic*, dan masih banyak lagi. Fitur utama dari MediaPipe ini adalah *landmark points extraction* dari deteksi terhadap benda yang sudah disebutkan sebelumnya.

MediaPipe Holistic Pipeline mengintegrasikan model terpisah untuk komponen pose, wajah, dan tangan, yang masing-masing dioptimalkan untuk domain khusus mereka. Pertama, diperkirakan pose dari tubuh manusia dengan detektor pose BlazePose dan *model landmark* berikutnya. Kemudian, dengan menggunakan landmark pose yang disimpulkan, diperoleh tiga *region of interest* (ROI) untuk masing-masing tangan dan wajah, dan menggunakan *model re-*

crop untuk meningkatkan ROI. . Semua landmark digabungkan dengan model pose untuk menghasilkan 540+ *landmark* penuh. (Google LLC, 2020)

2.2.8 Webcam

Webcam atau *web camera* adalah kamera yang memberikan gambar atau video secara real time melalui jaringan komputer. *Webcam* cenderung menggunakan resolusi yang kecil guna mempermudah dan mempercepat transmisi gambar ataupun video.

Webcam juga dapat digunakan untuk sesi obrolan video dengan orang lain. Alih-alih menyiarkan video di Web, pengguna dapat mengatur sesi obrolan video dengan satu atau lebih teman dan melakukan percakapan dengan audio dan video langsung. (Christensson, 2008)

2.2.9 Text-To-Speech

Text-to-speech (TTS) adalah jenis teknologi bantu yang membaca teks digital dengan keras. Ini kadang-kadang disebut teknologi "baca dengan lantang". TTS bekerja dengan hampir semua perangkat digital pribadi, termasuk komputer, ponsel cerdas, dan tablet. Semua jenis *file* teks dapat dibacakan, termasuk dokumen Word dan Pages. Bahkan halaman web online dapat dibaca dengan keras.

Suara dalam TTS dihasilkan oleh komputer, dan kecepatan membaca biasanya dapat dipercepat atau diperlambat. Kualitas suara bervariasi, tetapi beberapa suara terdengar seperti manusia. (WETA, 2000)

2.2.10 OpenCV

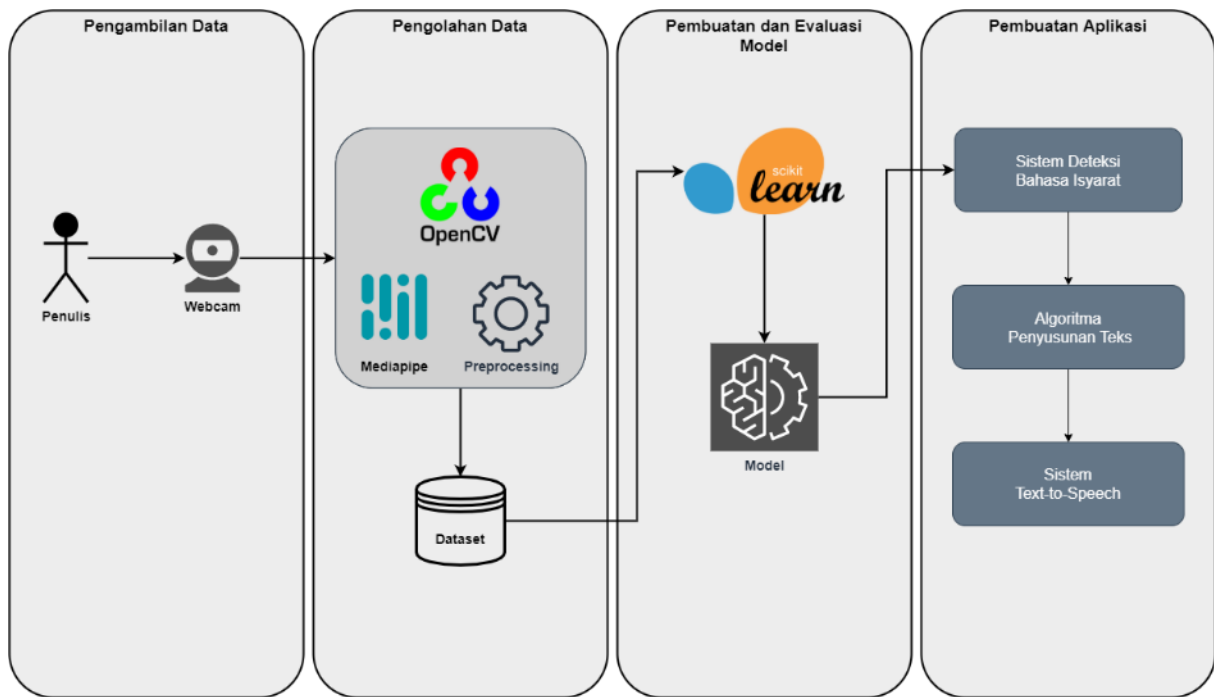
OpenCV (*Open Source Computer Vision Library*) adalah *library open-source* untuk *computer vision* dan *machine learning*. OpenCV dibangun untuk menyediakan infrastruktur yang umum digunakan untuk aplikasi yang menggunakan *computer vision* dan untuk mempercepat penggunaan persepsi mesin dalam produk komersial.

OpenCV mendukung bahasa pemrograman C++, Python, Java, dan lain sebagainya. OpenCV juga dapat digunakan di berbagai macam sistem operasi seperti Windows, Linux OS, OSX, Android, iOS, dan lain sebagainya. (Intel Corporation, 2022)

BAB III METODOLOGI

3.1 Metode yang digunakan

Metode yang digunakan dalam penelitian ini tersusun dari 4 tahap, pengambilan data, pengolahan data, pembuatan dan evaluasi *model*, dan diakhiri oleh pembuatan aplikasi. Urutan dan *outline* proses setiap tahap pada metode dapat dilihat pada Gambar 3.1.



Gambar 3.1 Diagram Metode Penelitian

3.1.1 Metode Pengambilan Data

Pada penelitian ini digunakan dataset *inhouse*, dataset sendiri, dimana *dataset* dibuat selama masa penelitian dilakukan, data yang diambil berupa *live video* yang langsung diproses menjadi *dataset*. Dapat dilihat pada Gambar 3.2 untuk tampilan yang terlihat saat pengambilan data.

Data diambil melalui *webcam* dengan bantuan *openCV* sebagai *library* pendukung *computer vision*. Data disini berupa gambar setiap *frame* dari *live feed* video yang dilakukan secara *real-time*. Untuk setiap bahasa isyarat akan dilakukan pengambilan data sebanyak 35 data dimana setiap datanya berupa gambar sebanyak 20 *frame* dengan resolusi 1280px * 720px, data ini akan diproses menjadi *dataset*.

Environment pengambilan data adalah sebagai berikut:

- Kondisi ruangan diterangi oleh lampu TL LED 36W 220V – 50Hz berposisi diatas langit – langit.
- Jarak tubuh dengan *webcam* internal laptop adalah 75cm.
- Layar laptop dan badan tegak lurus.

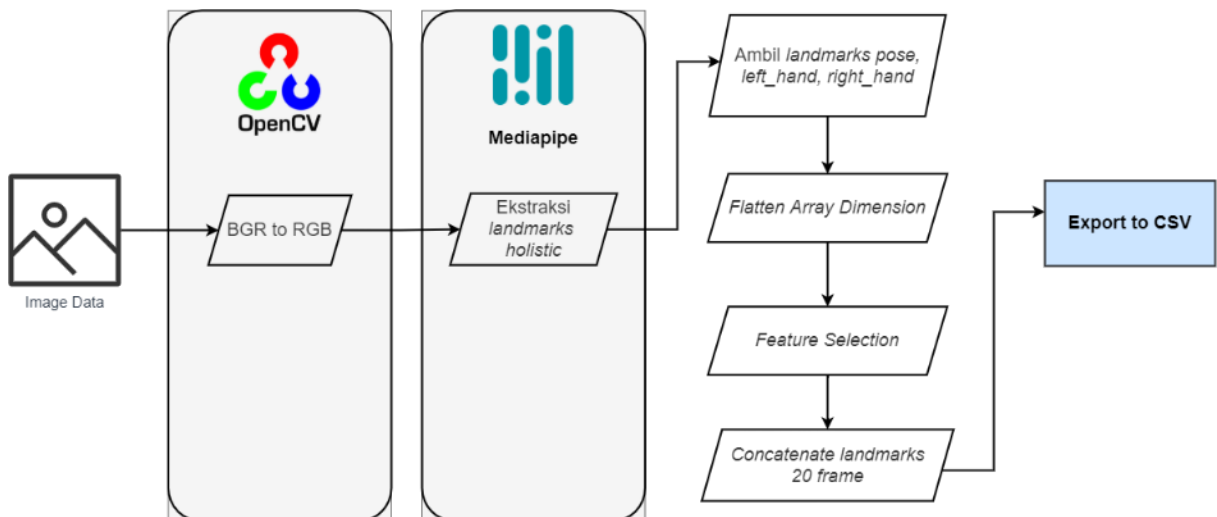
- *Webcam* menangkap tubuh dari rambut hingga siku tangan, posisi tubuh berada tepat di tengah – tengah *frame*.
- Tidak menggunakan *green screen* atau *background* berwarna polos.



Gambar 3.2 Pengambilan Data Menggunakan Webcam

3.1.2 Metode Pengolahan Data

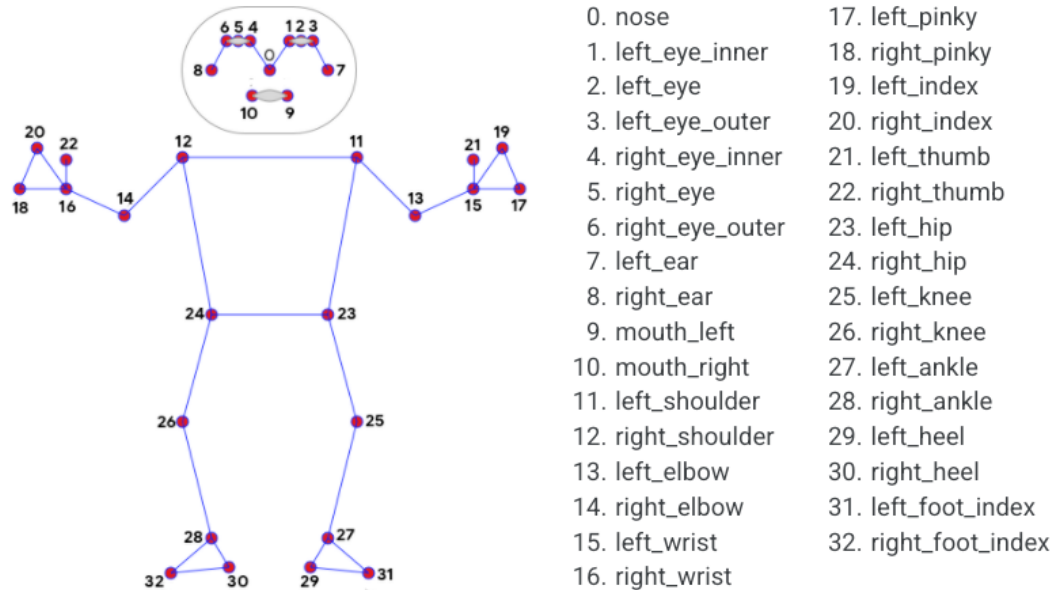
Pembuatan *dataset* dilakukan dengan melakukan pemrosesan pada setiap data yang telah didapatkan sebelumnya, 35 data setiap isyarat, lalu seluruh data akan disimpan pada *file .csv* (*Comma Separated Values*). Dapat dilihat pada Gambar 3.3 untuk diagram proses yang dilakukan dalam metode pengolahan data.



Gambar 3.3 Diagram Pembuatan *Dataset*

Gambar yang didapatkan diubah dari format warna BGR menjadi RGB agar dapat diproses oleh MediaPipe. Setelah itu akan didapatkan hasil berupa *landmarks* holistics MediaPipe, diambil hanya *pose_landmarks*, *left_hand_landmarks*, dan *right_hand_landmarks*. Koordinat x, y, dan z disimpan pada *array* lalu di *flatten* agar dimensi menjadi 1 dimensi, untuk

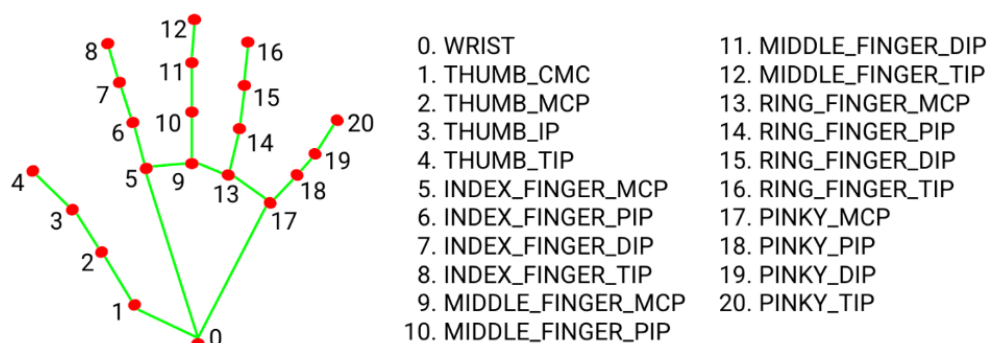
pose_landmark ada tambahan 1 fitur bernama *visibility* berisi *value* yang menentukan seberapa tampak *landmark* tersebut. Dilakukan *feature selection* terhadap *pose landmarks* sehingga hanya diambil sampai poin 16 (*right_wrist*) pada referensi Gambar 3.4. Proses pengambilan *landmarks* diulangi sebanyak 20 *frame* sebelum akhirnya *array* di ekspor menjadi *.csv*.



Gambar 3.4 Pose Landmarks MediaPipe (Google LLC, 2020)

Poin 17 sampai 32 pada *pose_landmark* pada referensi Gambar 3.4 tidak digunakan karena tidak terlihat pada *webcam* saat penggunaan aplikasi.

Gambar 3.5 adalah poin – poin *landmarks* pada kedua tangan yang dideteksi oleh MediaPipe. Terdiri dari 21 poin mulai dari *wrist* hingga *pinky_tip*.



Gambar 3.5 Hand Landmarks MediaPipe (Google LLC, 2020)

Didapatkan hasil akhir *dataset* dengan spesifikasi sebagai berikut:

- Memiliki 1 kolom '*class*' dimana values yang tersimpan adalah arti dari bahasa isyarat itu. Fitur ini dijadikan label pada *dataset*. '*class*' didefinisikan terlebih dahulu dan ditambahkan diawal sebagai kolom pertama pada *dataset*.

- Memiliki 3880 kolom yang terdiri dari koordinat x , y , z , dan *visibility* dari setiap poin *landmarks* pada *pose_landmarks*, *left_hand_landmarks*, dan *right_hand_landmarks* dengan kolom *visibility* hanya terdapat pada *pose_landmarks* setiap *frame* sebanyak 20 *frame*.

Bahasa isyarat BISINDO yang diimplementasikan pada aplikasi adalah:

- Huruf abjad: a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z.
- Kata ganti: aku, kamu, dia, kita.
- Kata ganti keluarga: ibu, ayah, kakak, adik, kakek, nenek, tante, om.
- Kata tanya: siapa, apa, kapan, berapa, dimana, kenapa, bagaimana.
- Kata kerja: makan, minum, tidur, baca, main.
- Benda: telepon, kacamata.
- Transportasi: motor, mobil, kereta, bus, pesawat.
- Konjungsi: dan, ke, di, dari.
- Kata: marah, bahagia, lelah, sehat, semangat, rumah, isyarat, nama, disini, asal, jangan, sudah, belum, tahu, teman.

Dibuat *class* 'IDLE' dan 'DELETE' dengan menggunakan gestur yang tidak ada di bahasa isyarat BISINDO. Kedua *class* ini akan digunakan untuk menjalankan fungsi *text-to-speech* dan penghapusan 1 *class* terakhir yang terdeteksi dalam aplikasi.

3.1.3 Metode Pembuatan dan Evaluasi Model

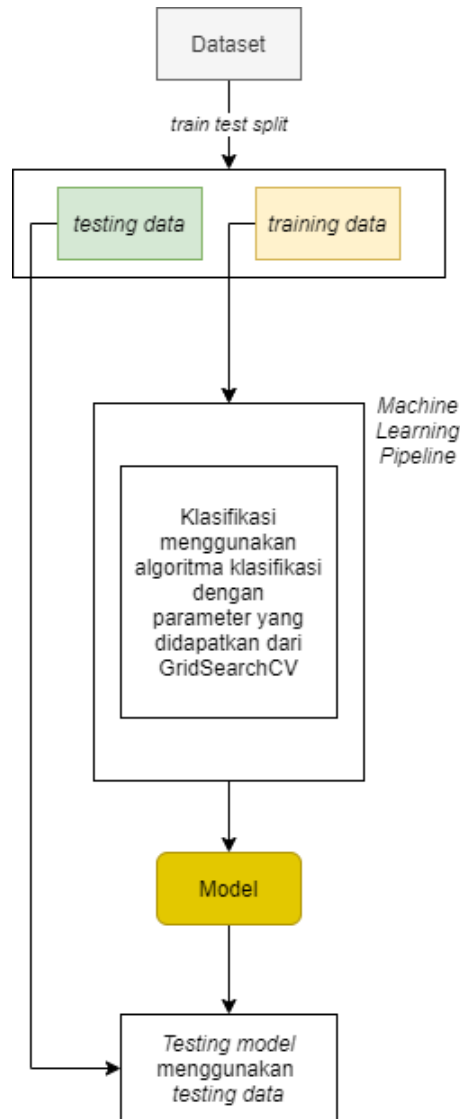
Dalam penelitian ini akan digunakan algoritma Support Vector Machine dalam pembuatan model, algoritma Random Forest dan K-Nearest Neighbor juga akan dicoba sebagai perbandingan. Akan dilakukan *hyperparameter tuning* menggunakan GridSearchCV milik scikit-learn, GridSearchCV adalah teknik untuk mencari nilai parameter terbaik dari kumpulan *parameter grid* yang diberikan. Ini pada dasarnya adalah metode *cross-validation*. Keluaran dari GridSearchCV ini adalah kombinasi parameter dengan nilai terbaik (akurasi), untuk menemukan parameter dengan tingkat akurasi terbaik untuk algoritma yang digunakan. Setelah parameter terbaik didapatkan dari *hyperparameter tuning*, akan digunakan parameter itu dalam pembuatan *model*.

Hyperparameter tuning akan dilakukan pada parameter berikut:

- Support Vector Machine
 - C , parameter regularisasi pada algoritma SVM digunakan untuk mengkalibrasi *model* untuk meminimalkan *loss function* yang disesuaikan dan mencegah *overfitting* atau *underfitting*.
 - *kernel*, untuk menentukan kernel yang digunakan pada algoritma SVM.
 - γ , parameter ini menentukan seberapa jauh pengaruh satu *training example*, dengan nilai rendah berarti 'jauh' dan nilai tinggi berarti 'dekat'.
 - *degree*, parameter ini mengatur derajat fungsi dari kernel polinomial (poly).
- Random Forest
 - *min_samples_split*, parameter ini mengatur jumlah minimal sampel yang diperlukan untuk membagi (*split*) *internal node*.

- *min_samples_leaf*, parameter ini mengatur jumlah minimal sampel yang diperlukan pada *leaf node*. Titik *split* pada kedalaman berapa pun hanya akan dipertimbangkan jika meninggalkan setidaknya sampel *training min_samples_leaf* di setiap cabang kiri dan kanan.
- *criterion*, parameter ini menentukan fungsi untuk mengukur kualitas sebuah *split*.
- *bootstrap*, parameter ini menentukan apakah *bootstrap sample* digunakan saat membuat pohon. Jika *False*, seluruh *dataset* digunakan untuk membangun setiap pohon.
- K-Nearest Neighbor
 - *n_neighbors*, parameter untuk menentukan jumlah neighbor.
 - *weights*, parameter ini memiliki 2 *value*, *uniform*: Semua titik di setiap lingkungan diberi bobot yang sama. *distance*: tetangga yang lebih dekat dengan titik data akan memiliki pengaruh yang lebih besar daripada tetangga yang lebih jauh.

Dataset dibagi menggunakan *train_test_split* dengan rasio data *training* sebesar 0,80 dan data *testing* sebesar 0,20. Akan digunakan parameter tambahan *random_state* pada Random Forest dan Support Vector Machine agar hasil *training* dapat direplika. *Pipeline* akan digunakan untuk mempermudah proses *training model*. *Dataset* yang digunakan berjumlah 2.730 data, 35 data untuk setiap '*class*'. Dengan rasio yang sudah ditentukan sebelumnya, sebanyak 546 data akan digunakan untuk *testing* dan 2.184 untuk *training model*. *Accuracy_score* akan digunakan untuk penilaian akurasi tiap *model*. Setelah *model* berhasil dibuat, digunakan library *pickle* untuk menyimpan *model* agar dapat diimpor kedalam aplikasi. Diagram pembuatan dan evaluasi *model* dapat dilihat pada Gambar 3.6.



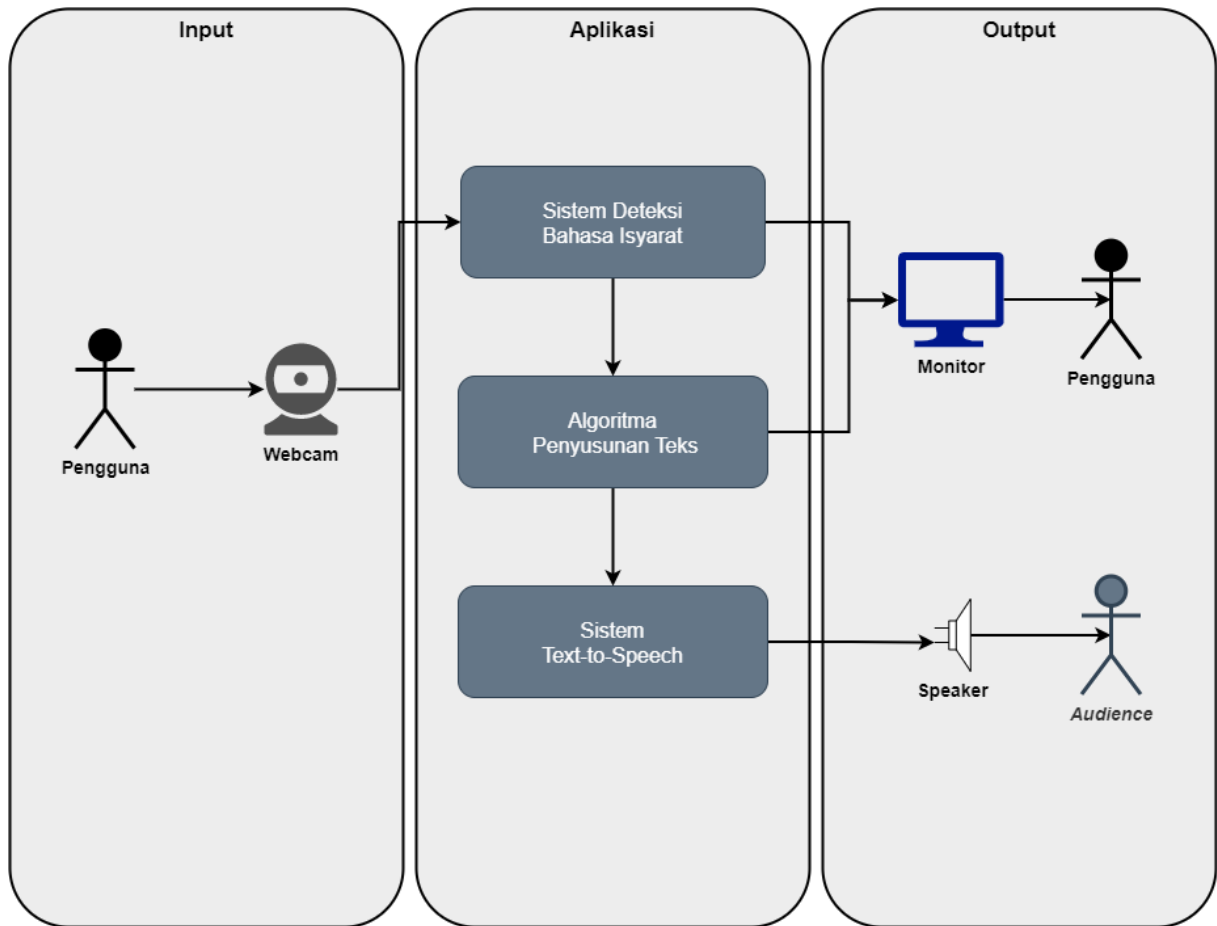
Gambar 3.6 Diagram Pembuatan Model

Penelitian ini menggunakan *library* scikit-learn yang tersedia untuk python dalam melakukan pembagian *dataset* menjadi *training data* dan *testing data*. *Library* tersebut juga akan digunakan untuk melakukan klasifikasi dengan algoritma dan parameter yang sudah didapatkan dari *hypertuning parameter*.

Dataset yang digunakan dalam penelitian ini akan dibuat selama penelitian dengan jumlah 30 data setiap bahasa isyarat (label). Dataset berbentuk nilai floating point koordinat x , y , z , dan *visibility* untuk setiap *landmark*, dimana *landmark* adalah titik – titik tertentu pada tubuh manusia yang dapat dilihat pada Gambar 3.4 dan Gambar 3.5.

3.1.4 Metode Pembuatan Aplikasi

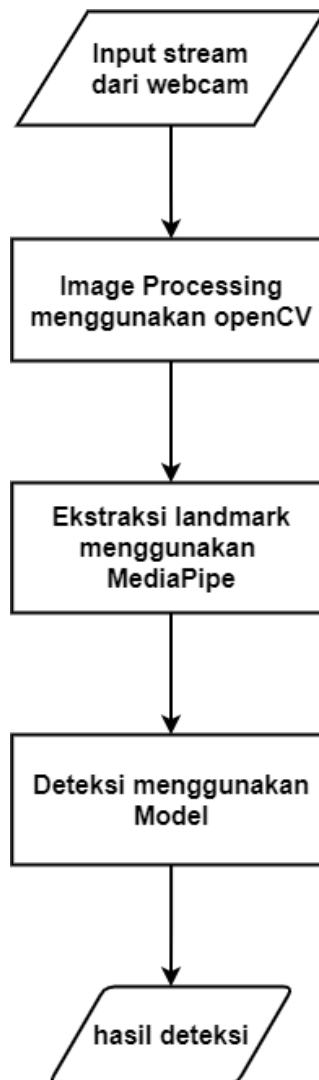
Aplikasi dibagi menjadi 3 bagian utama, yakni sistem deteksi bahasa isyarat, algoritma penyusunan teks dan sistem *text-to-speech*. Menerima *input* menggunakan webcam dan menghasilkan *output* berupa tampilan pada monitor pengguna dan suara melalui *speaker*. Diagram aplikasi dapat dilihat pada Gambar 3.7.



Gambar 3.7 Diagram Aplikasi

Input berupa gambar hasil tangkapan kamera saat pengguna menggunakan aplikasi secara *real-time* akan diproses oleh program agar dapat dilakukan prediksi menggunakan *model*. Data hasil prediksi akan diproses menggunakan algoritma yang diterapkan oleh pengguna untuk mengurangi kemungkinan terjadinya *false positive*, setelah itu fungsi penyusunan teks dijalankan. Fitur *text-to-speech* akan dijalankan ketika pengguna menggunakan gestur 'IDLE' yang menandakan pengguna sedang tidak berisyarat.

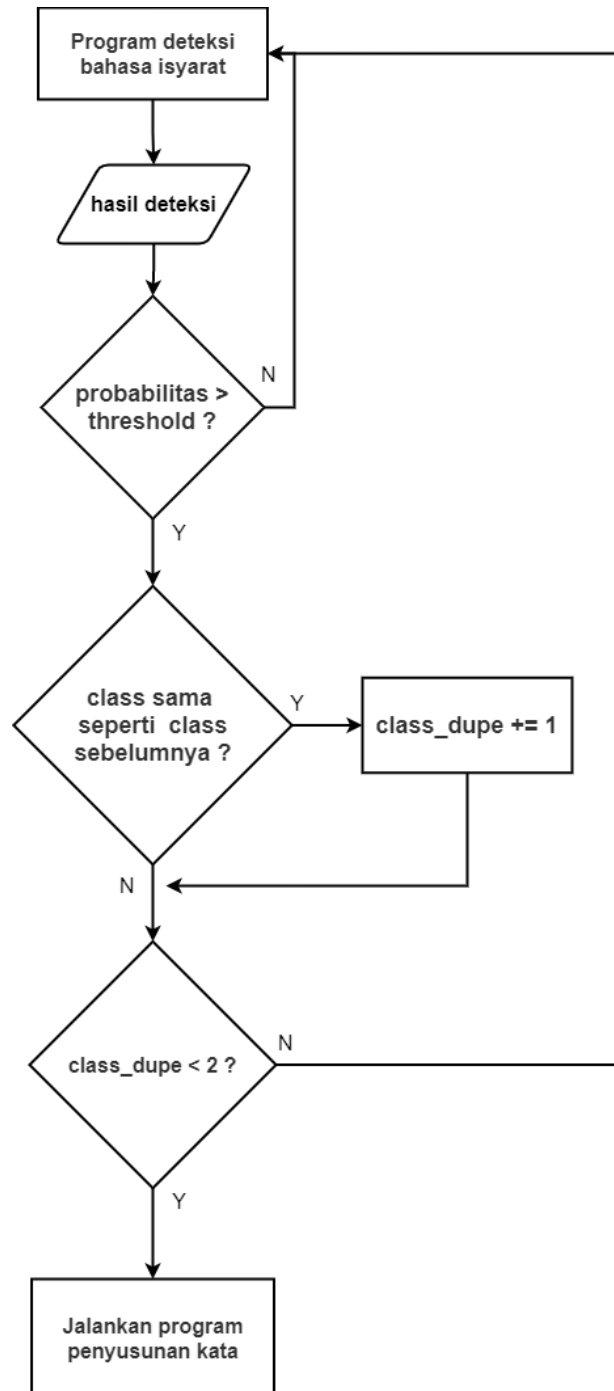
Bagian pertama dalam aplikasi ini adalah sistem deteksi bahasa isyarat dimana aplikasi akan mendeteksi dan memprediksi bahasa isyarat yang dilakukan oleh pengguna. *Flowchart* sistem ini dapat dilihat pada Gambar 3.8.



Gambar 3.8 Sistem Deteksi BISINDO

Aplikasi menggunakan openCV untuk mengakses dan mengambil *input* menggunakan *webcam* internal laptop, *input* berupa *livestream video* pengguna yang akan berisyarat. Video diproses setiap *frame* menggunakan openCV lalu akan dilakukan ekstraksi *landmarks* menggunakan MediaPipe, proses ini diulangi sebanyak 20 kali (*20 frame processing*), hasil ekstraksi *landmarks* akan disimpan kedalam variabel. Setiap setelah 20 *frame*, akan dilakukan prediksi bahasa isyarat BISINDO menggunakan variabel yang berisi *landmarks* oleh *model*. Hasil prediksi ini akan diteruskan ke tahap selanjutnya.

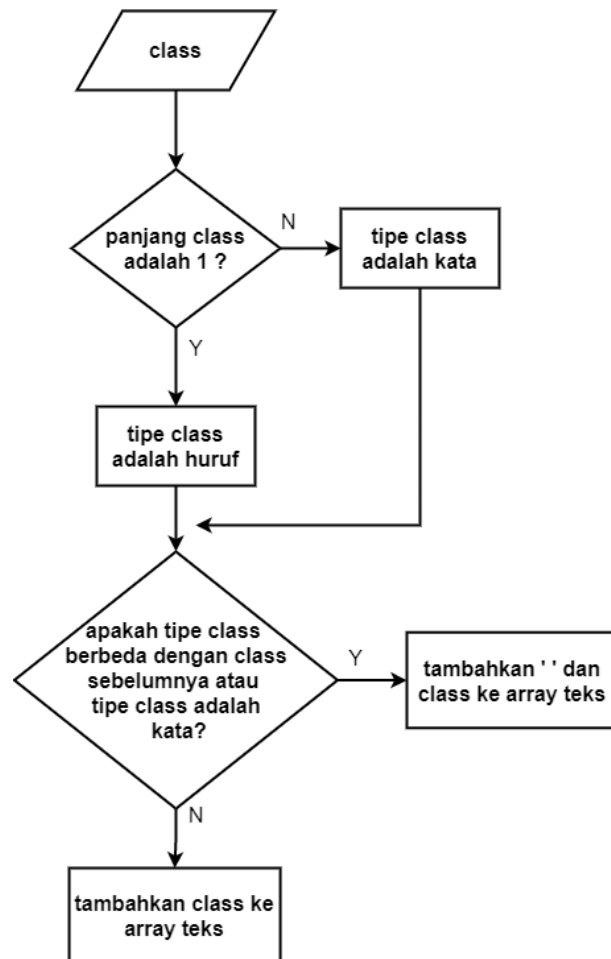
Flowchart dari algoritma untuk mengurangi kemungkinan terjadinya *false positive* dapat dilihat pada Gambar 3.9. Hasil deteksi atau *class* yang diperoleh sebelumnya diperiksa jika probabilitasnya melebihi threshold, setelah itu program akan mengecek jika *class* sama seperti *class* hasil deteksi sebelumnya, jika iya maka nilai *class_dupe* bertambah 1. Program mengecek jika *class_dupe* kurang dari 2, jika iya maka hasil deteksi akan dilanjutkan ke program penyusunan kata.



Gambar 3.9 Algoritma Pengurangan *False Positive Detection*

Dalam aplikasi ini juga akan ditambahkan fitur *delete* untuk menghapus hasil deteksi terakhir yang tersimpan ke dalam *array* teks untuk mengatasi *false positive* dan salah isyarat oleh pengguna. Fitur ini dapat diaktifkan dengan cara menutupi *webcam* menggunakan tangan agar aplikasi tidak mendeteksi keberadaan manusia (pengguna).

Program penyusunan teks berfungsi untuk mempermudah pembacaan kalimat yang disusun oleh pengguna. Program ini akan membedakan jika *class* adalah sebuah huruf atau sebuah kata, lalu akan ditambahkan spasi (‘ ’) diantara kata. *Flowchart* deteksi tipe *class* dan penyusunan teks dapat dilihat pada Gambar 3.10.

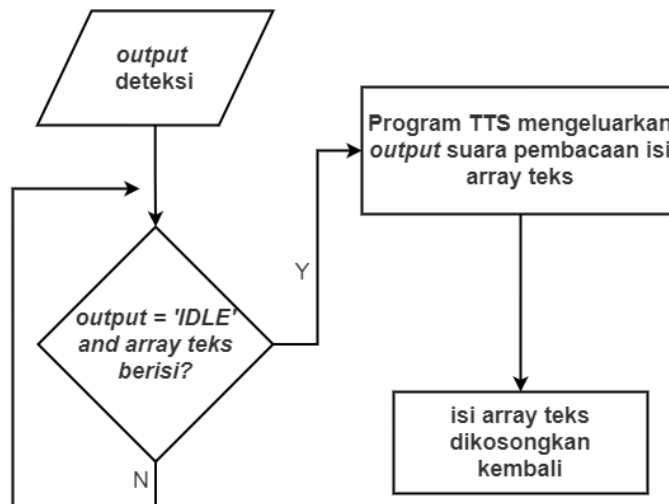


Gambar 3.10 Penyusunan Teks

Data hasil prediksi akan ditampilkan seperti berikut :

- *Overlay* teks yang disusun dari *array* teks pada jendela yang menunjukkan gambar hasil tangkapan *webcam*.
- *Overlay* hasil prediksi (*class* dan *probability*) tersebut pada jendela yang menunjukkan gambar hasil tangkapan *webcam*.

Setelah deteksi BISINDO dan penyusunan teks dilakukan, maka pengguna dapat menggunakan fitur TTS agar aplikasi dapat mengeluarkan suara untuk didengar. Flowchart aktivasi TTS dapat dilihat pada Gambar 3.11.



Gambar 3.11 Algoritma *Text-to-Speech*

Program akan melakukan pengecekan apakah hasil prediksi bernilai 'IDLE' dan *array* teks memiliki isi. Jika iya maka program akan menjalankan *text-to-speech* untuk menghasilkan *output* berupa *audio* pembacaan isi dari *array* melalui *speaker* laptop.

3.2 Bahan dan Peralatan yang Digunakan

Diperlukan beberapa bahan dan peralatan agar penelitian ini dapat dilakukan dan berjalan dengan lancar.

3.2.1 Laptop

Laptop disini memiliki fungsi sebagai berikut:

- Sebagai *webcam* untuk mendapatkan data (berupa *live feed gambar*) dari pengguna.
- Sebagai *audio device* untuk mengeluarkan *speech*.
- Sebagai *platform* pembuatan aplikasi.
- Sebagai *hardware* untuk melakukan proses pembuatan model machine learning.

Spesifikasi laptop yang digunakan adalah sebagai berikut:

- Processor: AMD Ryzen 5 Mobile 3500U 2.10 GHz
- RAM: 8 GB DDR4 2400MHz
- GPU: Integrated Radeon Vega 8 Mobile Gfx 2GB Integrated Memory
- Resolusi *Webcam*: 720p HD
- Sistem Operasi: Windows 10 Home 64bit
- *Audio Device*: SonicMaster

3.2.2 Dataset Bahasa Isyarat BISINDO

Dataset yang digunakan disini adalah *dataset* bahasa isyarat BISINDO berupa kumpulan koordinat 3 dimensi dari *landmarks* solusi holistics MediaPipe. *Dataset* digunakan

untuk *training* klasifikasi *model machine learning* dan *testing* terhadap *model* untuk mendeteksi dan mengartikan bahasa isyarat BISINDO.

3.2.3 *Software* Pendukung

Dalam penelitian ini akan digunakan beberapa *software* sebagai berikut:

- Anaconda Navigator.
- Jupyter Notebook.
- Bahasa pemrograman Python beserta *library* pendukung.
- Sistem Operasi Windows 10 Home 64bit.

3.3 Urutan Pelaksanaan Penelitian

Penelitian memiliki urutan pelaksanaan penyusunan proposal tugas akhir, studi literatur dan pengumpulan data, desain sistem dan implementasi, dan diakhiri dengan pengujian dan evaluasi.

3.3.1 Penyusunan Proposal Tugas Akhir

Penyusunan proposal tugas akhir yang terdiri dari BAB pendahuluan, tinjauan pustaka, dan metodologi. Pada proposal tugas akhir juga dilengkapi dengan lampiran seperti daftar pustaka, dan sebagainya.

3.3.2 Studi Literatur dan Pengumpulan Data

Dalam mengerjakan tugas akhir ini diperlukan untuk melakukan studi terhadap literatur yang sudah ada sebelumnya sebagai acuan dan referensi yang akan digunakan dalam tugas akhir ini. Survei akan dilakukan ke beberapa narasumber untuk mengumpulkan data yang bisa membantu dalam implementasi.

3.3.3 Desain Sistem dan Implementasi

Pada tahap ini, desain sistem dan implementasi akan dilakukan sesuai perencanaan sebelumnya. Sistem yang akan dibuat adalah aplikasi yang dapat menerjemahkan bahasa isyarat BISINDO menjadi teks dan dari teks tersebut akan dikeluarkan *output* berupa *speech* menggunakan teknologi *text-to-speech*(TTS).

3.3.4 Pengujian dan Evaluasi

Setelah sistem selesai dibuat akan dilakukan pengujian dan evaluasi. Akan dilakukan 2 pengujian yakni pengujian menggunakan dataset dan pengujian menggunakan video *feed* secara *real-time* dari pengguna.

Untuk pengujian pertama akan menggunakan *metrics* akurasi saat melakukan training model, akan diperoleh skor akurasi dari model. Kriteria yang diperhatikan adalah tingkat akurasi. Pengujian kedua adalah secara *real-time* pengguna akan memperagakan bahasa isyarat didepan kamera. Kriteria pengujian adalah jarak pengguna dari kamera, kondisi pencahayaan ruangan.

3.3.5 Penyusunan Laporan Tugas Akhir

Setelah semua tahap penelitian selesai dilakukan, laporan mengenai hasil penelitian yang sudah dilakukan akan disusun.

BAB IV HASIL DAN PEMBAHASAN

4.1 Implementasi

Hasil implementasi akan dibagi menjadi beberapa sub-bab yakni dataset yang digunakan yang menjelaskan tentang *dataset*, instalasi dan persiapan fitur *speech*, *pseudocode*, dan implementasi fitur aplikasi.

4.1.1 Dataset Yang Digunakan

Dataset yang digunakan berjumlah 2.730 data, dimana masing – masing label memiliki 35 data. Total kata yang diimplementasikan, termasuk huruf abjad dan 2 label tambahan ‘IDLE’ dan ‘DELETE’, berjumlah 78 label. *dataset* sudah berlabel, kolom ‘*class*’, dan memiliki fitur berupa koordinat *landmarks* di setiap *frame* sebanyak 20 *frame* dalam nilai *float*. Koordinat sudah dinormalisasi ke [0.0, 1.0]. Dapat dilihat pada Tabel 4.1 untuk *preview* kolom dataset yang digunakan. Dataset yang digunakan dibuat selama penelitian berlangsung.

Dalam proses training *model* menggunakan rasio pembagian data *training* dan data *testing* sebesar 0,80 dan 0,20 makadari itu didapatkan 2.184 *training data* dan 546 *testing data*.

Tabel 4.1 *Preview Dataset BISINDO*

	class	frame1_pose_x0	frame1_pose_y0	frame1_pose_z0	frame1_pose_v0	frame1_pose_x1	frame1_pose_y1	frame1_pose_z1	frame1_pose_v1	frame
0	IDLE	0.479768	0.234315	-0.870053	0.999969	0.497794	0.191298	-0.834720	0.999906	
1	IDLE	0.481666	0.258503	-0.797928	0.999935	0.497719	0.209137	-0.762619	0.999827	
2	IDLE	0.484274	0.233368	-0.789906	0.999960	0.501494	0.191627	-0.749877	0.999897	
3	IDLE	0.483406	0.233815	-0.715744	0.999956	0.501212	0.191544	-0.680201	0.999891	
4	IDLE	0.494860	0.247808	-0.864271	0.999927	0.511851	0.202900	-0.829192	0.999822	
...	...	...	...	...	...	...	...	...	...	...
2725	DI	0.459727	0.268375	-0.509997	0.999979	0.479165	0.213399	-0.466049	0.999962	
2726	DI	0.464861	0.268548	-0.481229	0.999985	0.483445	0.216548	-0.439693	0.999973	
2727	DI	0.467012	0.258367	-0.436137	0.999986	0.485768	0.207595	-0.389492	0.999976	
2728	DI	0.470859	0.264778	-0.402169	0.999989	0.489868	0.214206	-0.360957	0.999981	
2729	DI	0.467895	0.271438	-0.455640	0.999984	0.487186	0.223277	-0.413844	0.999974	

2730 rows × 3881 columns

Dilakukan pengecekan apakah semua class terdapat pada *dataset* menggunakan Kode Sumber 4.1:

```
isyarat_dict =
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', '
U', 'V', 'W', 'X', 'Y', 'Z',
  'IDLE', 'DELETE',
  'AKU', 'KAMU', 'DIA', 'KITA',
  'SIAPA', 'APA', 'KAPAN', 'DIMANA', 'BAGAIMANA', 'BERAPA', 'KENAPA',
  'IBU', 'AYAH', 'KAKAK', 'ADIK', 'KAKEK', 'NENEK', 'TANTE', 'OM',
  'MARAH', 'LELAH', 'SEHAT', 'SEMANGAT', 'BAHAGIA',
  'MOTOR', 'MOBIL', 'BUS', 'KERETA', 'PESAWAT',
  'RUMAH',
  'ISYARAT',
  'MAKAN', 'MINUM', 'TIDUR', 'BACA',
```

```

        'TELEPON', 'KACAMATA',
        'NAMA', 'DISINI', 'ASAL', 'JANGAN', 'SUDAH', 'BELUM', 'TAHU', 'MAIN',
        'TEMAN', 'DAN', 'KE', 'DI', 'DARI']
missing = []
for i in isyarat_dict:
    if not i in df['class'].unique():
        missing.append(i)
print("Class yang tidak terdapat pada dataset :", missing)

```

Kode Sumber 4.1 Kode Pengecekan *Missing Class*

Dibuat *dictionary* yang berisi semua bahasa isyarat yang diimplementasikan dalam aplikasi ini, setelah itu untuk setiap *value* pada *dictionary* tersebut dilakukan pengecekan pada *dataset* yang sudah dimuat dalam bentuk *dataframe*. Untuk setiap *value* yang tidak ditemukan, akan disimpan dalam *array* missing lalu ditampilkan. Hasil *running* kode dapat dilihat pada Gambar 4.1.



```

In [5]: 1 isyarat_dict = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
2                 'IDLE', 'DELETE',
3                 'AKU', 'KAMU', 'DIA', 'KITA',
4                 'SIAPA', 'APA', 'KAPAN', 'DIMANA', 'BAGAIMANA', 'BERAPA', 'KENAPA',
5                 'IBU', 'AYAH', 'KAKAK', 'ADIK', 'KAKEK', 'NENEK', 'TANTE', 'OM',
6                 'MARAH', 'LELAH', 'SEHAT', 'SEMANGAT', 'BAHAGIA',
7                 'MOTOR', 'MOBIL', 'BUS', 'KERETA', 'PESAWAT',
8                 'RUMAH',
9                 'ISYARAT',
10                'MAKAN', 'MINUM', 'TIDUR', 'BACA',
11                'TELEPON', 'KACAMATA',
12                'NAMA', 'DISINI', 'ASAL', 'JANGAN', 'SUDAH', 'BELUM', 'TAHU', 'MAIN',
13                'TEMAN', 'DAN', 'KE', 'DI', 'DARI']

In [6]: 1 #Cek jika ada class yang missing
2 missing = []
3 for i in isyarat_dict:
4     if not i in df['class'].unique():
5         missing.append(i)
6 print("Class yang tidak terdapat pada dataset :", missing)

Class yang tidak terdapat pada dataset : []

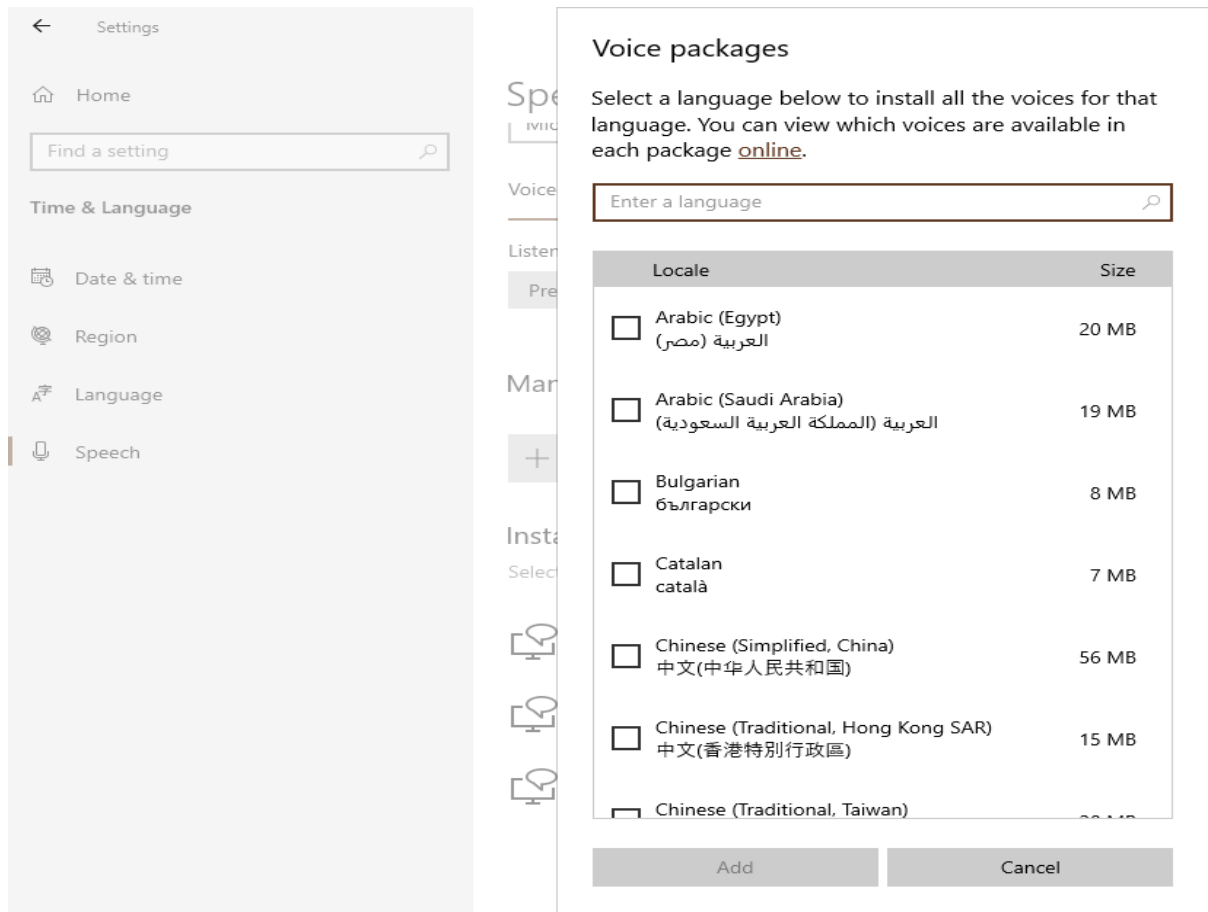
```

Gambar 4.1 Hasil *Running* Kode Pengecekan *Missing Class*

Berdasarkan hasil running kode yang dapat dilihat pada Gambar 4.1 dapat dipastikan bahwa tidak ada *class*(bahasa isyarat) yang tidak ada dalam *dataset*.

4.1.2 Instalasi dan Persiapan Fitur *Speech*

Untuk bisa menggunakan *speech* bahasa Indonesia, perlu dilakukan instalasi terlebih dahulu pada sistem operasi Windows 10. Dalam *Settings*, pilih bagian *Speech*, lalu dibawah “*Manage voices*” terdapat opsi “*Add Voices*”. Tampilan pengaturan seperti pada Gambar 4.2.



Gambar 4.2 Add Voices

Setelah itu, *Add Voice packages* Indonesian (Bahasa Indonesia). Setelah proses unduh dan pasang selesai, *voice packages* bahasa Indonesia akan muncul pada daftar. Pada Gambar 4.3 terlihat bahwa *voice packages* bahasa Indonesia sudah terpasang.

Manage voices



Installed voice packages

Select a voice package to remove it.

	English (United States) English (United States)	48 MB
	Indonesian Bahasa Indonesia	7 MB
	Malay (Malaysia) Bahasa Melayu (Malaysia)	5 MB

Gambar 4.3 Voice Packages

Kode registry digunakan untuk memungkinkan *voice packages* bahasa Indonesia yang telah dipasang untuk dipakai di aplikasi pada penelitian ini, isi dari *file .reg* dapat dilihat pada Kode Sumber 4.2.

```
Windows Registry Editor Version 5.00

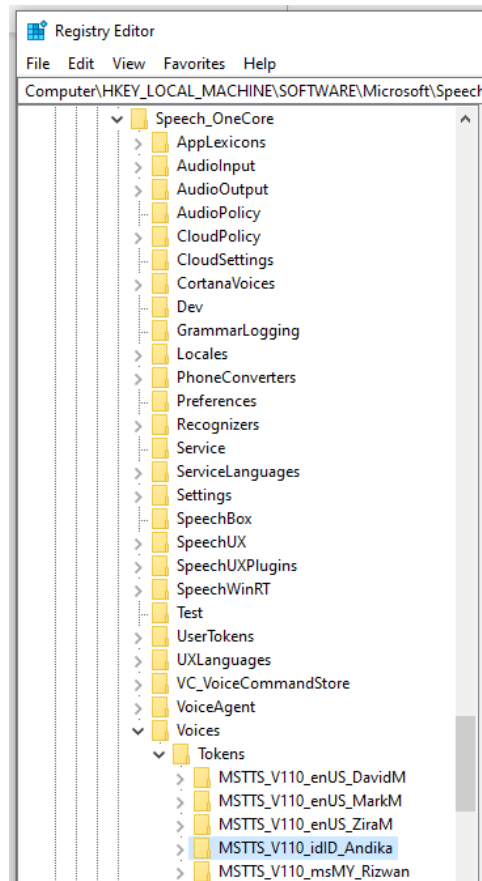
[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Speech\Voices\Tokens\MSTTS_V110_idID_Andika]
@="Microsoft Andika - Indonesian (Indonesia)"
"421"="Microsoft Andika - Indonesian (Indonesia)"
"CLSID"="{179F3D56-1B0B-42B2-A962-59B7EF59FE1B}"
"LangDataPath"=hex(2):25,00,77,00,69,00,6e,00,64,00,69,00,72,00,25,00,5c,00,53,\
00,70,00,65,00,65,00,63,00,68,00,5f,00,4f,00,6e,00,65,00,43,00,6f,00,72,00,\
65,00,5c,00,45,00,6e,00,67,00,69,00,6e,00,65,00,73,00,5c,00,54,00,54,00,53,\
00,5c,00,69,00,64,00,2d,00,49,00,44,00,5c,00,4d,00,53,00,54,00,54,00,53,00,\
4c,00,6f,00,63,00,49,00,64,00,49,00,44,00,2e,00,64,00,61,00,74,00,00,00
"VoicePath"=hex(2):25,00,77,00,69,00,6e,00,64,00,69,00,72,00,25,00,5c,00,53,00,\
70,00,65,00,65,00,63,00,68,00,5f,00,4f,00,6e,00,65,00,43,00,6f,00,72,00,65,\
00,5c,00,45,00,6e,00,67,00,69,00,6e,00,65,00,73,00,5c,00,54,00,54,00,53,00,\
5c,00,69,00,64,00,2d,00,49,00,44,00,5c,00,4d,00,31,00,30,00,35,00,37,00,41,\
00,6e,00,64,00,69,00,6b,00,61,00,00,00

[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Speech\Voices\Tokens\MSTTS_V110_idID_Andika\Attributes]
"Age"="Adult"
"DataVersion"="11.0.2016.1016"
"Gender"="Male"
"Language"="421"
"Name"="Microsoft Andika"
"SharedPronunciation"=""
"Vendor"="Microsoft"
"Version"="11.0"
```

Kode Sumber 4.2 *registry voice pack* Bahasa Indonesia

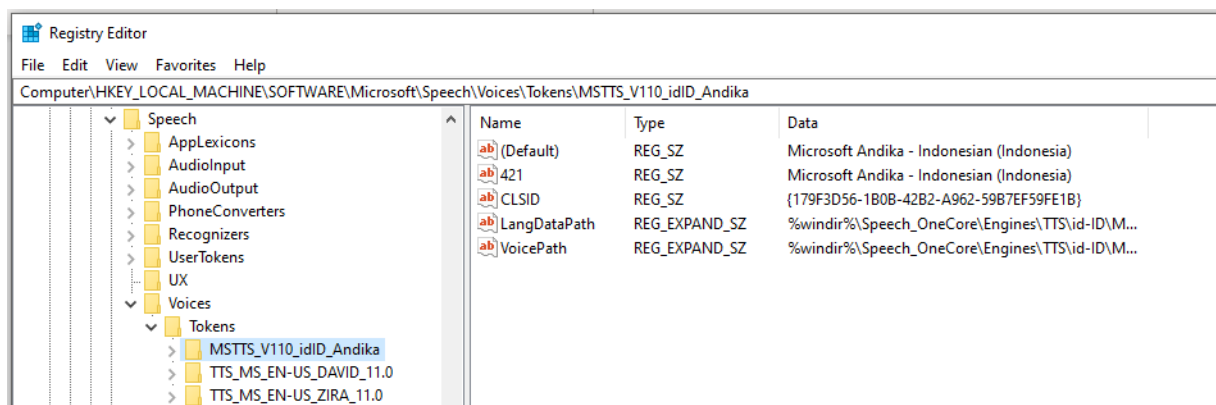
Kode sumber 4.2 disimpan dengan nama *file* 'MSTTS_V110_idID_Andika.reg'. Setelah itu *file* tersebut dijalankan untuk memungkinkan penggunaan *narrator* bahasa Indonesia dalam implementasi aplikasi. Tahap ini dilakukan karena setelah instalasi awal, *voice pack* masih belum tersedia atau belum bisa digunakan oleh program selain Microsoft Speech itu sendiri.

Voice pack hanya tersedia di *directory* /Speech_OneCore yang dimana program tidak dapat mengakses. Dapat dilihat pada gambar 4.4 *directory* lengkap *voice pack*.



Gambar 4.4 Registry Sebelum Fix

Voice pack berada di *directory* /Speech dimana bisa diakses melalui python. Dapat dilihat pada gambar 4.5 *directory* lengkap *voice pack*.



Gambar 4.5 Registry Sesudah Fix

4.1.3 Implementasi Fitur Aplikasi

Hasil implementasi dari fitur aplikasi yang terdiri dari deteksi BISINDO, penyusunan kalimat, fitur *delete*, fitur *text-to-speech*.

4.1.3.1 Deteksi BISINDO

Deteksi BISINDO adalah bagian pertama dalam aplikasi dimana sistem akan melakukan proses terhadap gambar dan mendeteksi bahasa isyarat yang dilakukan pengguna.

Pada Kode Sumber 4.3 gambar terlebih dahulu diubah menjadi format warna RGB dari agar bisa diproses menggunakan MediaPipe. Hal ini dikarenakan format gambar yang diambil oleh OpenCV secara *default* menggunakan format BGR dimana format tersebut tidak dapat di proses oleh MediaPipe.

```
image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
image.flags.writeable = False
results = holistic.process(image)
image.flags.writeable = True
image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
```

Kode Sumber 4.3 *Preprocessing Image Format*

Pada Kode Sumber 4.4 hasil prosesi gambar, ekstraksi *landmark*, oleh MediaPipe akan diambil dan disimpan di variabel yang sesuai. Dalam program ditambahkan pengecekan jika pose tubuh, tangan kanan, tangan kiri tidak terdeteksi pada gambar maka koordinat untuk *landmarks* bagian tersebut akan diisi dengan nilai 0.0 agar tidak terjadi *missing value*, proses ini diulangi selama gambar yang diproses belum melebihi 21. Setelah itu semua koordinat akan di *concatenate* menjadi satu *array*.

```
try:
    if frame_counter < 21:
        if not results.pose_landmarks:
            pose_row = [0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
                        0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
                        0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
                        0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,
                        0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,]
        else:
            pose = results.pose_landmarks.landmark
            pose_row = list(np.array([[landmark.x, landmark.y, landmark.z,
                                       landmark.visibility] for landmark in pose]).flatten())
            pose_row = pose_row[0:68]
        if not results.right_hand_landmarks:
            rhand_row = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
        else:
            rhand = results.right_hand_landmarks.landmark
            rhand_row = list(np.array([[landmark.x, landmark.y, landmark.z] for
                                       landmark in rhand])).flatten())
        if not results.left_hand_landmarks:
            lhand_row = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                           0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                           0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
```

```

0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

else:
    lhand = results.left_hand_landmarks.landmark
    lhand_row = list(np.array([[landmark.x, landmark.y, landmark.z] for
    landmark in lhand])).flatten())

row += pose_row+lhand_row+rhand_row
frame_counter += 1

```

Kode Sumber 4.4 *Feature Selection* dan Persiapan Variabel Untuk Deteksi

Dilakukan pengecekan jika gambar yang telah diproses sebanyak 20. *Array* selanjutnya akan diubah menjadi *dataframe* dengan penamaan fitur. *Dataframe* ini yang akan diprediksi oleh *model*, hasil prediksi adalah *class* dan nilai probabilitasnya. Kode dapat dilihat pada Kode Sumber 4.5.

```

if frame_counter == 20:
    frame_counter = 0
    X = pd.DataFrame([row])
    X.columns = xcolumnname
    row = []

    sl_predicted_class = model.predict(X)[0]
    sl_class_prob = model.predict_proba(X)[0]
    sl_class_prob = str(round(sl_class_prob[np.argmax(sl_class_prob)],2))

```

Kode Sumber 4.5 Kode Prediksi BISINDO

4.1.3.2 Penyusunan Kalimat dan Fitur *Delete*

Setelah sistem deteksi BISINDO, bagian selanjutnya pada aplikasi adalah penyusunan kalimat hasil deteksi agar dapat dibaca oleh pengguna dan sistem TTS.

Dilakukan pengecekan jika nilai probabilitas dari prediksi melebihi *threshold* yang ditentukan, jika kondisi tersebut dipenuhi maka akan dilakukan pengecekan jika class hasil deteksi saat ini sama seperti hasil deteksi sebelumnya. Kode dapat dilihat pada Kode Sumber 4.6.

```

if float(sl_class_prob) > prob_min_threshold:
    any_detection = True
    if sl_predicted_class_temp == sl_predicted_class and sl_predicted_class !=
    'IDLE' and sl_predicted_class != 'DELETE':
        class_dupe += 1
    else:
        class_dupe = 0

```

Kode Sumber 4.6 Pengecekan *Threshold* dan *Class Dupe*

Dilakukan pengecekan jika class hasil deteksi bernilai '*DELETE*' dan *array* teks memiliki isi. Jika kedua syarat ini terpenuhi maka program akan menjalankan fungsi *delete* untuk

menghapus hasil deteksi terakhir pada isi *array* teks. Kode untuk fitur delete ini dapat dilihat pada Kode Sumber 4.7.

```
elif sl_predicted_class == 'DELETE' and len(text_list) > 0:
    del text_list[-1]
    if text_list[-1:][0] == ' ':
        del text_list[-1]
    sl_predicted_class_temp = text_list[-1:][0]
    if class_dupe > 0:
        class_dupe -= 1
    if len(text_list[-1:][0]) == 1:
        last_detected_type = 'huruf'
    else:
        last_detected_type = 'kata'
```

Kode Sumber 4.7 Fitur *Delete*

Program akan mengecek jika *class_dupe*, variabel untuk mengecek berapa kali *class* yang sama terdeteksi, jika bernilai kurang dari 2. Jika kondisi memenuhi maka hasil deteksi akan ditentukan apakah termasuk huruf abjad atau kata untuk menentukan jika pengguna sedang mengeja atau melakukan isyarat kata dengan cara menghitung panjang dari *class* hasil deteksi. Kode untuk melakukan pengecekan ini adalah Kode Sumber 4.8.

```
elif class_dupe < 2 and sl_predicted_class != 'IDLE' and sl_predicted_class != 'DELETE':
    sl_predicted_class_temp = sl_predicted_class
    if len(sl_predicted_class) == 1:
        detected_type = 'huruf'
    else:
        detected_type = 'kata'
```

Kode Sumber 4.8 Pengecekan Tipe *Class*

Setelah tipe deteksi ditentukan maka program akan menyusun kalimat menggunakan hasil deteksi, jika bertipe huruf maka hasil deteksi akan digabungkan ke *array* teks tanpa menambahkan spasi (menandakan bahwa pengguna sedang mengeja), jika bertipe kata maka hasil deteksi akan digabungkan serta ditambahkan spasi ke *array* teks. Kode untuk penyusunan ini adalah Kode Sumber 4.9.

```
if last_detected_type == 'start':
    text_list.append(sl_predicted_class)
    last_detected_type = detected_type
elif detected_type != last_detected_type or detected_type == 'kata':
    text_list.append(' ')
    text_list.append(sl_predicted_class)
    last_detected_type = detected_type
else:
    text_list.append(sl_predicted_class)
    last_detected_type = detected_type
```

Kode Sumber 4.9 Kode Penyusunan Kalimat pada *Array* Teks

4.1.3.3 Fitur *Text-to-Speech*

Salah satu fitur utama dari aplikasi ini adalah sistem *text-to-speech* atau biasa disebut TTS. Sistem ini berfungsi untuk membacakan, mengeluarkan output berupa audio, kalimat yang disusun oleh pengguna menggunakan bahasa isyarat.

Fungsi kode 4.10 adalah untuk import dan konfigurasi “narator” bahasa Indonesia yang akan membacakan teks dari program. Aplikasi menggunakan *value* 125 untuk konfigurasi *properties* ‘rate’ untuk menentukan *word per second* atau kecepatan membaca narator, nilai ini adalah rerata kecepatan membaca orang pada umumnya. Program akan melakukan iterasi untuk mengecek apakah narator Bahasa Indonesia sudah terpasang di *device* atau belum.

```
#Konfigurasi TTS Engine
tts_engine = pyttsx3.init("sapi5")
voices = tts_engine.getProperty('voices')
tts_engine.setProperty('rate',125)
for v in range(len(voices)):
    if voices[v].id ==
'HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Speech\Voices\Tokens\MSTTS_V110_idID_Andika
':
        id_voice = v

if id_voice:
    tts_engine.setProperty('voice', voices[id_voice].id)
    tts_status = True
    print('Speech Bahasa ditemukan dengan ID value', id_voice)
else:
    tts_status = False
    print('Speech Bahasa gagal ditemukan atau belum terpasang')
```

Kode Sumber 4.10 Kode Konfigurasi TTS

Jika *class* adalah ‘IDLE’ dan *array* teks memiliki isi *text-to-speech* akan dijalankan. *Array* teks akan diubah menjadi *string* lalu sistem TTS akan membuat narator membacakan *string* tersebut, setelah itu *array* teks dikosongkan. Selama narator masih berjalan, aplikasi berhenti melakukan deteksi dan *capture image* melalui *webcam*. Kode untuk aktivasi sistem TTS ini adalah Kode Sumber 4.11.

```
if sl_predicted_class == 'IDLE' and len(text_list) > 0:
    text_string = ''.join(text_list)
    tts_engine.say(text_string)
    tts_engine.runAndWait()
    text_list = []
    text_string,last_detected_type = '', 'start'
    sl_predicted_class_temp = 'temp'
    class_dupe = 0
```

Kode Sumber 4.11 Kode Aktivasi TTS

4.2 Hasil

Hasil berupa pemaparan hasil dari *hyperparameter tuning*, *model* dari *machine learning*, evaluasi kerja implementasi dalam *real-time*, dan fitur *text-to-speech*.

4.2.1 Hyperparameter Tuning

Tabel 4.2, Tabel 4.3, Tabel 4.4, Tabel 4.5 adalah hasil dari *hyperparameter tuning* untuk setiap algoritma, akan ditampilkan 10 hasil dengan skor tertinggi(*mean_test_score*).

Algoritma Random Forest

Tabel 4.2 Hasil *Hyperparameter Tuning* Random Forest

No.	bootstrap	criterion	min_samples_leaf	min_samples_split	mean_test_score
1	True	gini	1	2	0.984615
2	True	entropy	1	6	0.984249
3	True	gini	1	6	0.984249
4	True	entropy	3	2	0.983883
5	True	gini	1	10	0.983516
6	True	entropy	4	6	0.982784
7	False	entropy	1	2	0.982784
8	True	gini	4	2	0.982784
9	False	entropy	3	6	0.982784
10	True	gini	3	10	0.982418

Algoritma Support Vector Machine – *kernel: linear, rbf, sigmoid*

Tabel 4.3 Hasil *Hyperparameter Tuning* SVM

No.	C	gamma	kernel	mean_test_score
1	100	scale	rbf	0.980586
2	1000	scale	rbf	0.980220
3	1000	auto	sigmoid	0.978388
4	10	scale	linear	0.977656
5	1000	auto	rbf	0.977656
6	1000	auto	linear	0.977656
7	1000	scale	linear	0.977656
8	100	auto	linear	0.977656
9	100	scale	linear	0.977656
10	10	auto	linear	0.977656

Algoritma Support Vector Machine – *kernel: poly*

Tabel 4.4 Hasil *Hyperparameter Tuning* SVM *kernel poly*

No.	C	degree	gamma	kernel	mean_test_score
1	1000	3	scale	poly	0.971429
2	100	3	scale	poly	0.971062
3	1000	5	scale	poly	0.969231
4	100	5	scale	poly	0.969231
5	10	5	scale	poly	0.968132
6	10	3	scale	poly	0.967766
7	1000	7	scale	poly	0.961538

8	1000	3	auto	poly	0.961538
9	100	7	scale	poly	0.961538
10	10	7	scale	poly	0.960806

Algoritma K-Nearest Neighbor

Tabel 4.5 Hasil *Hyperparameter Tuning* KNN

No.	n_neighbors	weights	mean_test_score
1	1	uniform	0.942857
2	2	distance	0.942857
3	1	distance	0.942857
4	2	uniform	0.932967
5	3	uniform	0.927473
6	3	distance	0.927473
7	4	distance	0.927473
8	4	uniform	0.920147
9	6	distance	0.917582
10	5	distance	0.916850

4.2.2 Machine Learning Model

Didapatkan tingkat akurasi prediksi terhadap data *testing* sebesar 99.7% untuk metode Random Forest, 98.7% untuk metode Support Vector Machine, 96.5% untuk metode K-Nearest Neighbor. Hasil dapat dilihat pada Tabel 6.

Tabel 4.6 Hasil *Testing Machine Learning Model*

Algoritma	accuracy_score	Akurasi (Pembulatan)
Random Forest	0.9963369963369964	99.7%
Support Vector Machine	0.9871794871794872	98.7%
K-Nearest Neighbor	0.9652014652014652	96.5%

4.2.3 Evaluasi Kerja Aplikasi

Untuk evaluasi pengujian penggunaan aplikasi akan dilakukan beberapa pengujian dengan memperhatikan kriteria jarak (pengguna dengan kamera), kondisi pencahayaan ruangan.

Environment pengujian pertama adalah sebagai berikut:

- Kondisi ruangan terang (diterangi lampu TL LED berposisi di langit – langit).
- Jarak pengguna dengan *webcam* 75cm.
- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian pertama adalah sebagai berikut:

- Nama pengguna menggunakan bahasa isyarat huruf abjad.
- Kalimat “AKU SEMANGAT”.
- PESAWAT, BUS, MOTOR.
- IBU, TANTE, OM.

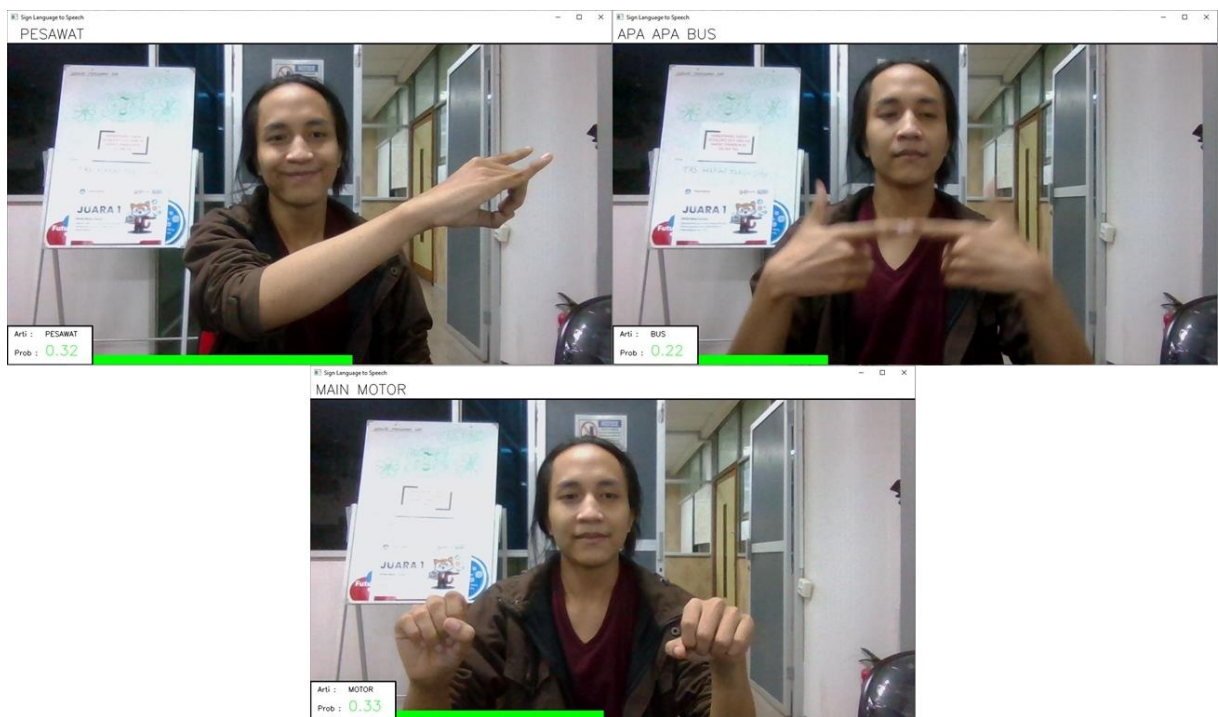
Tes pengguna 1 (Dindra):



Gambar 4.6 Pengujian 1 Nama Pengguna 1



Gambar 4.7 Pengujian 1 "AKU SEMANGAT" Pengguna 1

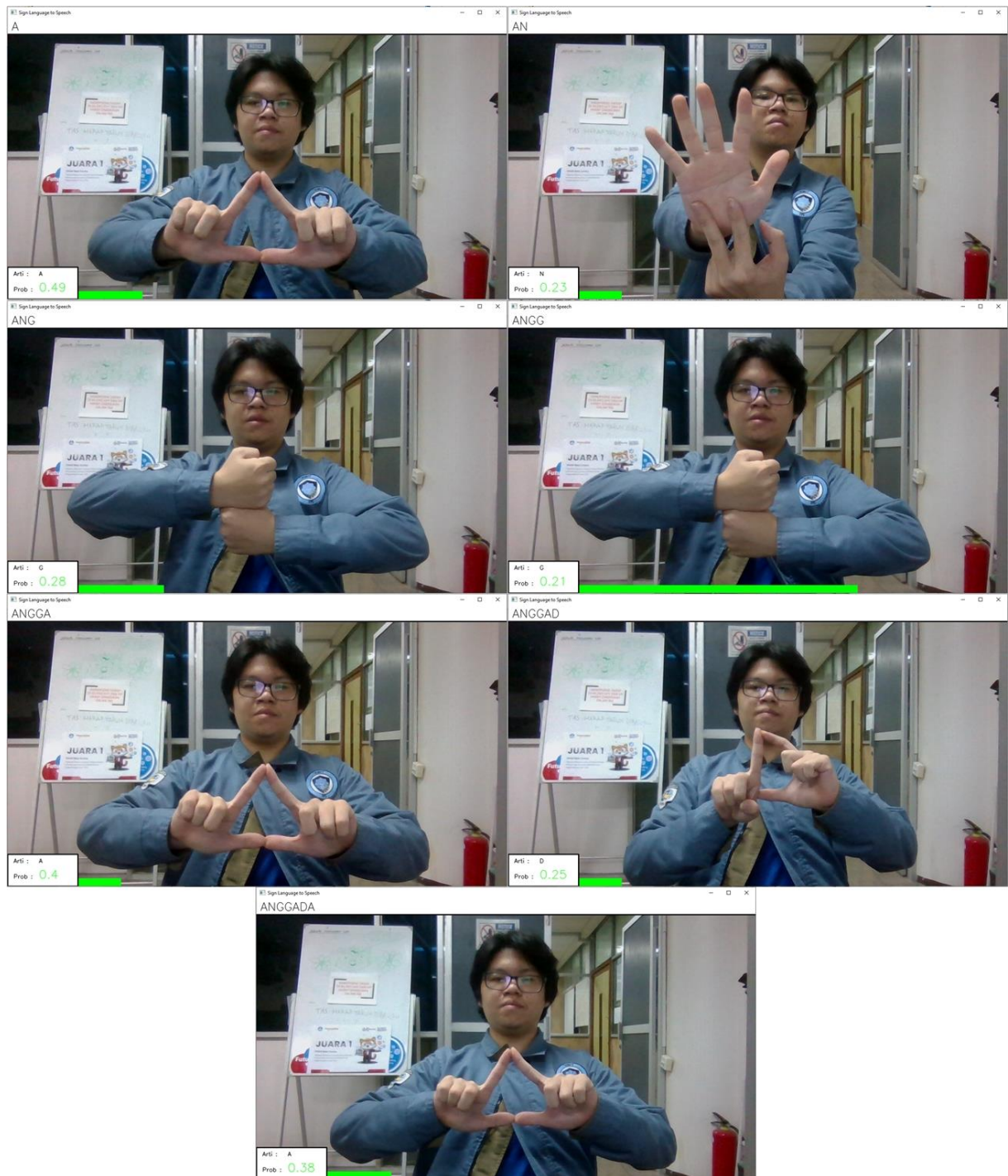


Gambar 4.8 Pengujian 1 Transportasi Pengguna 1



Gambar 4.9 Pengujian 1 Kata Ganti Keluarga Pengguna 1

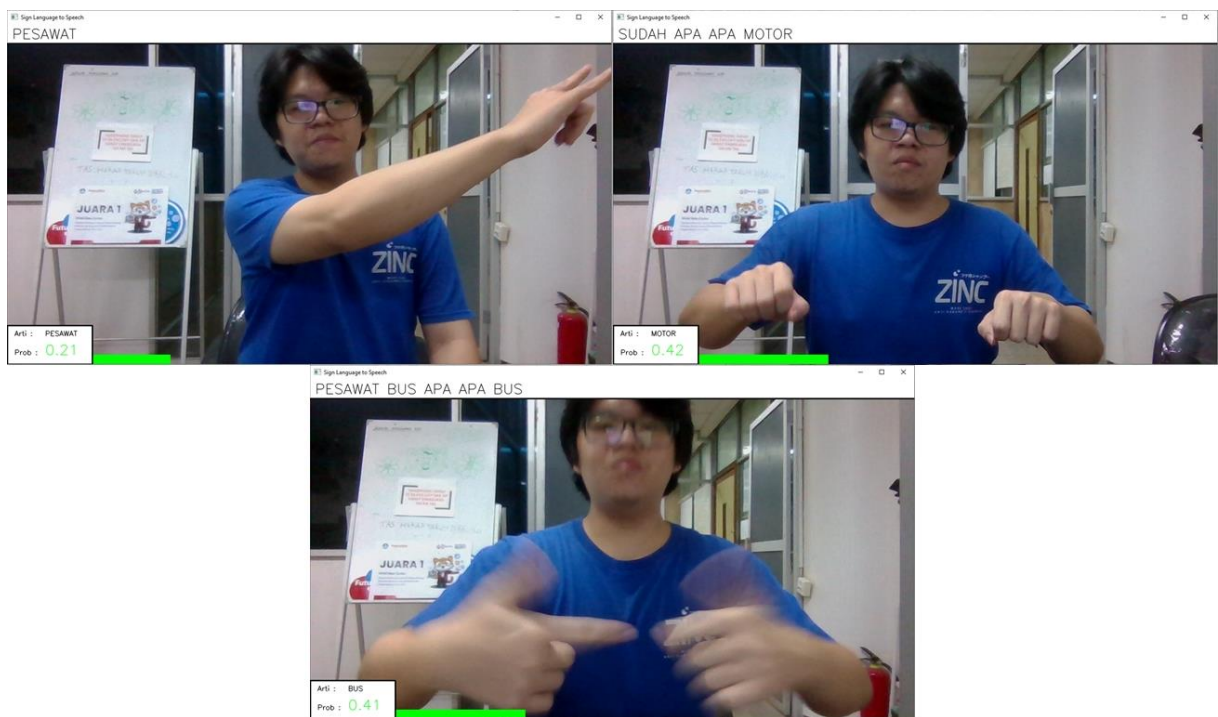
Tes pengguna 2 (Anggada):



Gambar 4.10 Pengujian 1 Nama Pengguna 2



Gambar 4.11 Pengujian 1 "AKU SEMANGAT" Pengguna 2



Gambar 4.12 Pengujian 1 Transportasi Pengguna 2



Gambar 4.13 Pengujian 1 Kata Ganti Keluarga Pengguna 2

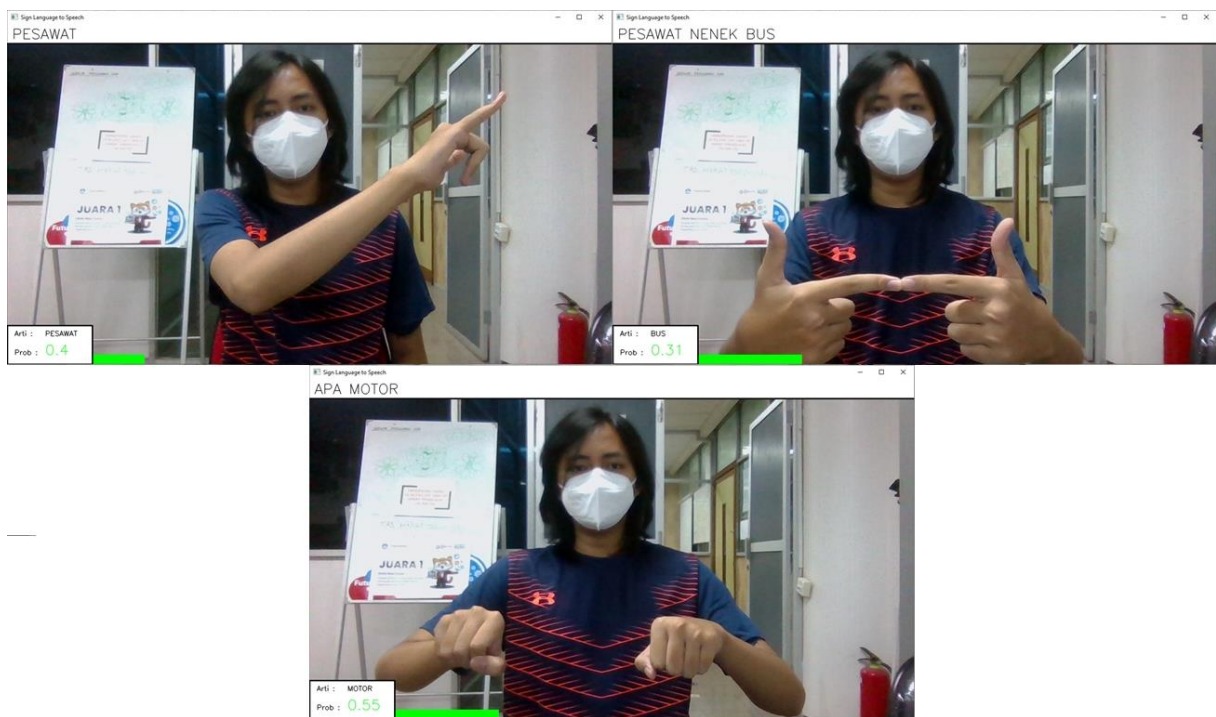
Tes pengguna 3 (Zulfikar):



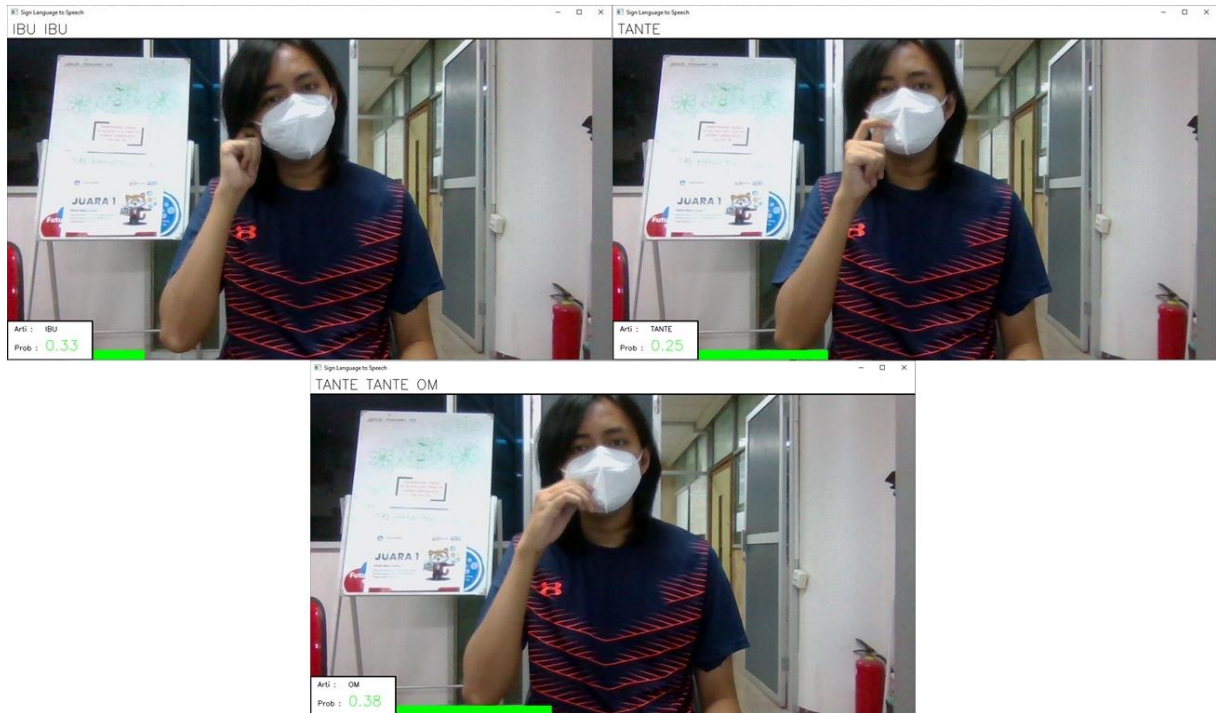
Gambar 4.14 Pengujian 1 Nama Pengguna 3



Gambar 4.15 Pengujian 1 "AKU SEMANGAT" Pengguna 3



Gambar 4.16 Pengujian 1 Transportasi Pengguna 3

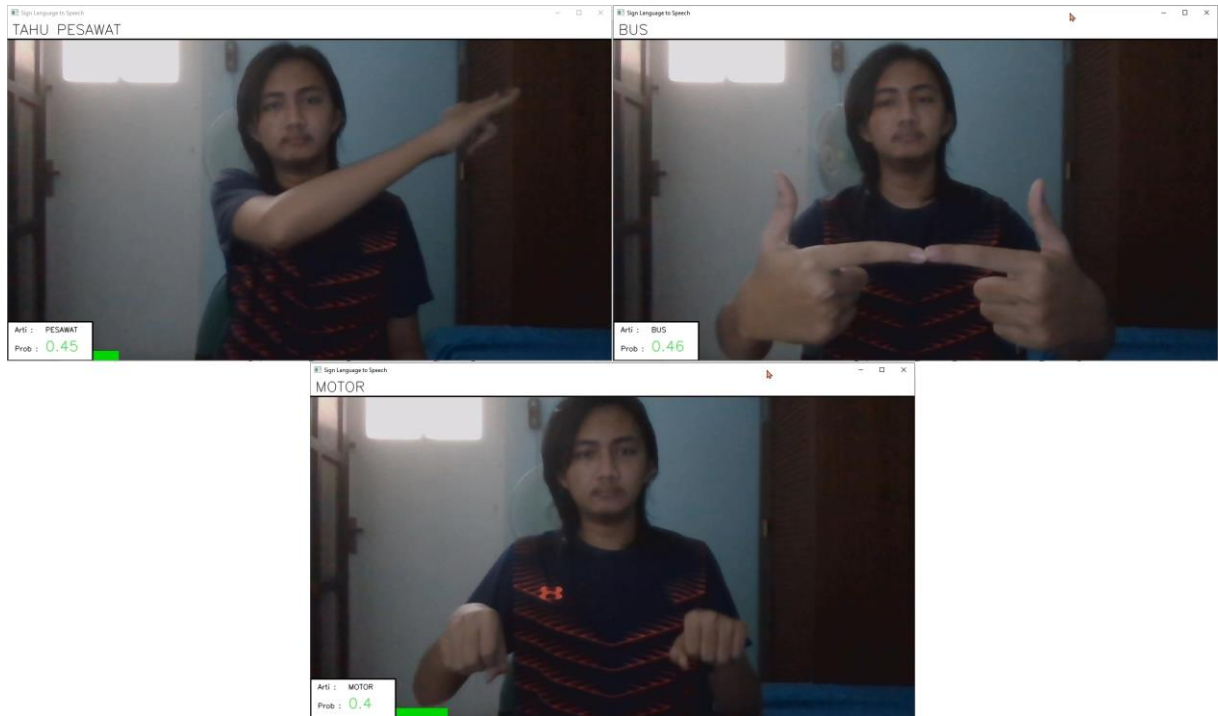


Gambar 4.17 Pengujian 1 Kata Ganti Keluarga Pengguna 3

Environment pengujian kedua adalah sebagai berikut:

- Kondisi pencahayaan ruangan redup (diterangi sinar matahari pada siang hari melalui celah di pintu masuk ruangan).
- Jarak pengguna dengan *webcam* 75cm.
- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian kedua adalah PESAWAT, BUS, MOTOR, IBU, TANTE, OM.



Gambar 4.18 Pengujian 2 “PESAWAT, BUS, MOTOR”



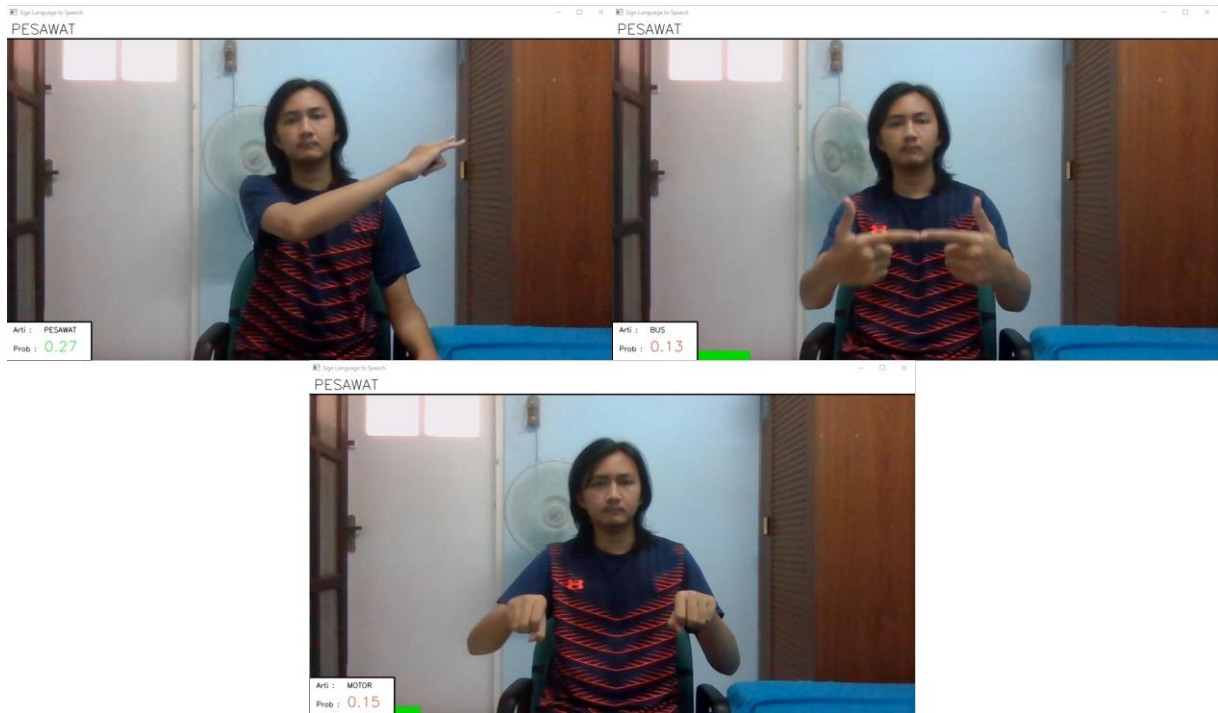
Gambar 4.19 Pengujian 2 “IBU, TANTE, OM”

Environment pengujian ketiga adalah sebagai berikut:

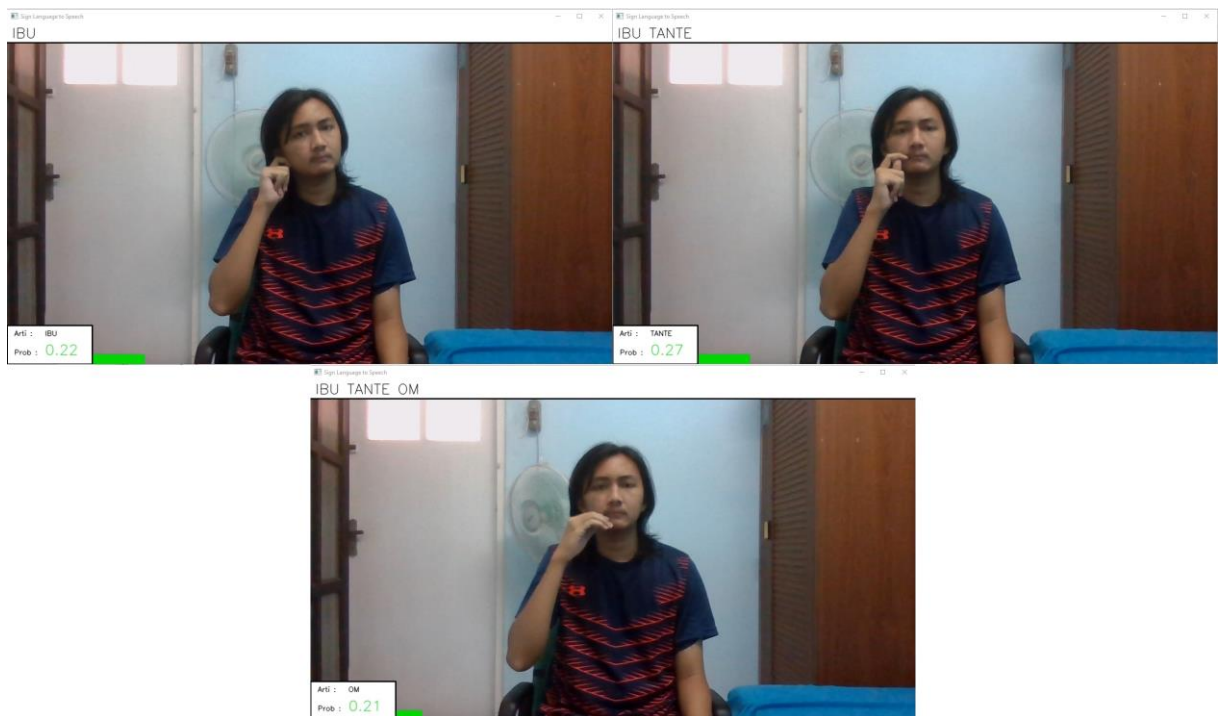
- Kondisi pencahayaan ruangan terang (diterangi oleh lampu TL LED 36W 220V – 50Hz berposisi diatas langit – langit.).
- Jarak pengguna dengan *webcam* 125cm.

- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian ketiga adalah PESAWAT, BUS, MOTOR, IBU, TANTE, OM.



Gambar 4.20 Pengujian 3 “PESAWAT, BUS, MOTOR”

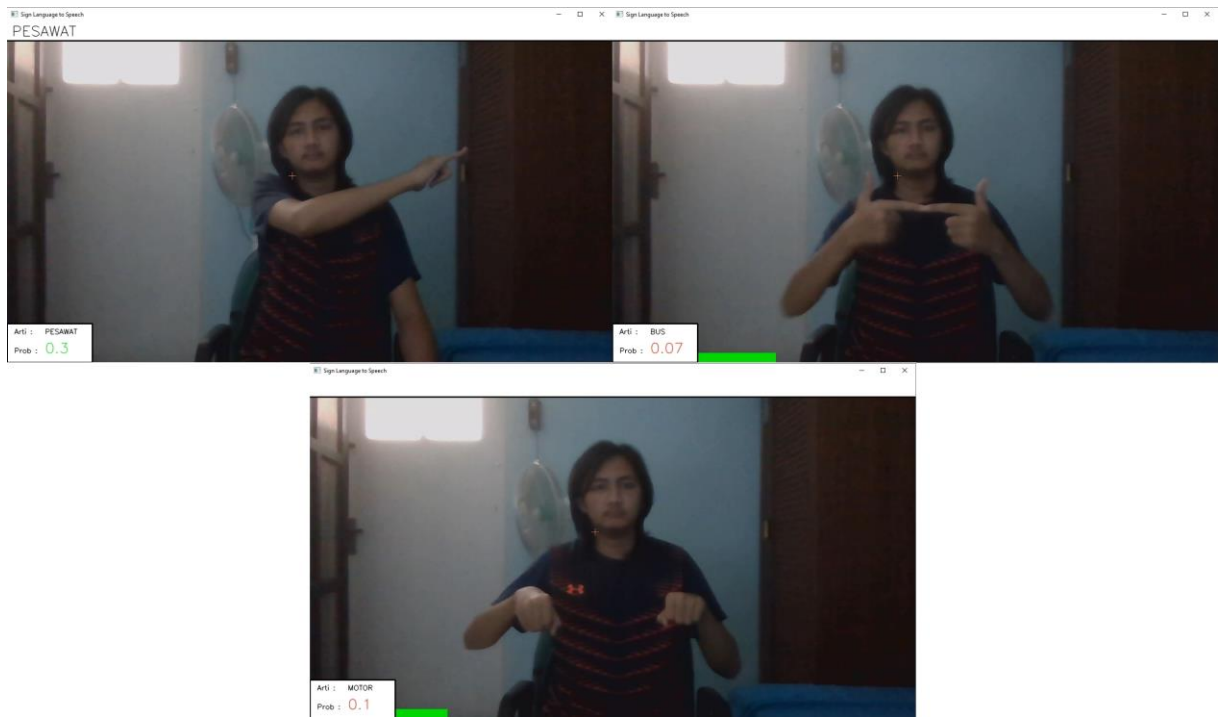


Gambar 4.21 Pengujian 3 “IBU, TANTE, OM”

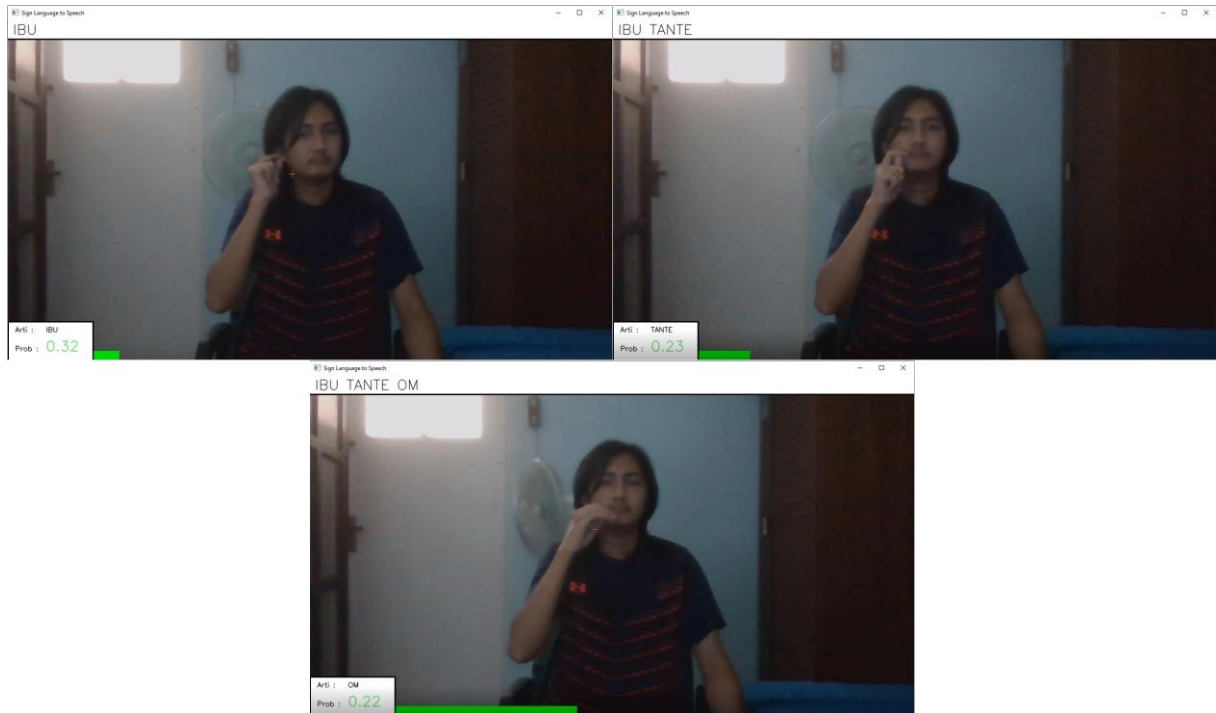
Environment pengujian keempat adalah sebagai berikut:

- Kondisi pencahayaan ruangan redup (diterangi sinar matahari pada siang hari melalui celah di pintu masuk ruangan).
- Jarak pengguna dengan *webcam* 125cm.
- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian keempat adalah PESAWAT, BUS, MOTOR, IBU, TANTE, OM.



Gambar 4.22 Pengujian 4 “PESAWAT, BUS, MOTOR”

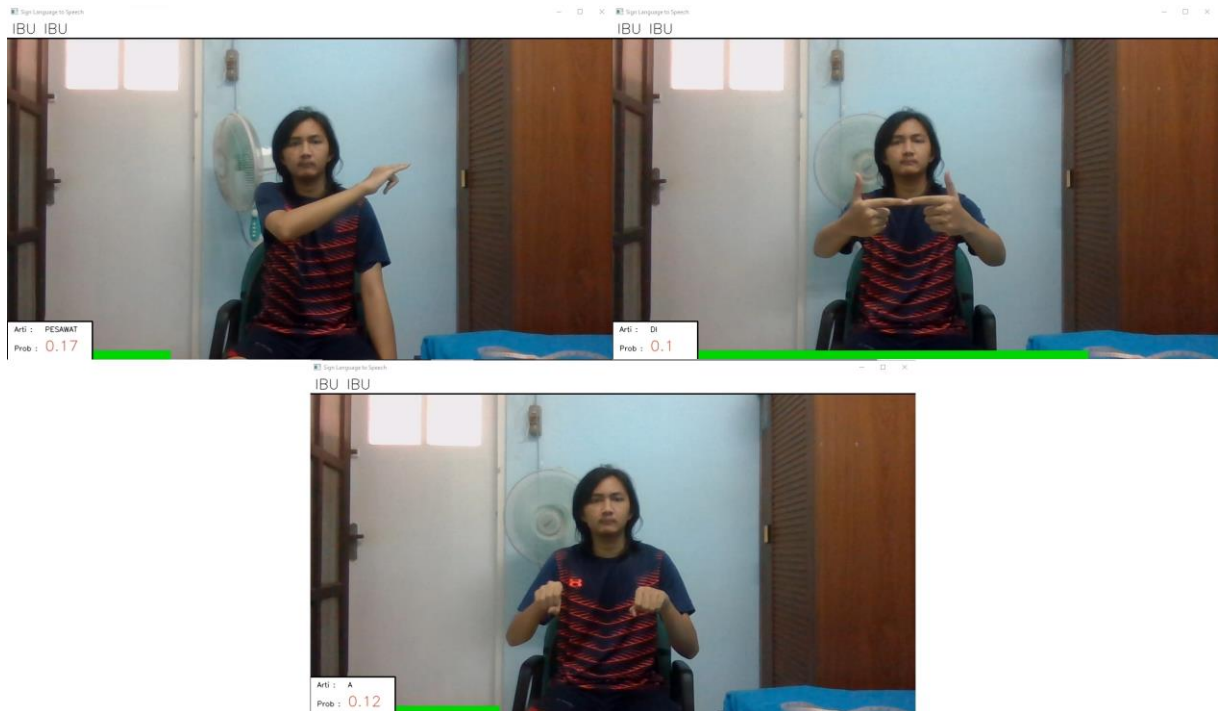


Gambar 4.23 Pengujian 4 “IBU, TANTE, OM”

Environment pengujian kelima adalah sebagai berikut:

- Kondisi pencahayaan ruangan terang (diterangi oleh lampu TL LED 36W 220V – 50Hz berposisi diatas langit – langit.).
- Jarak pengguna dengan *webcam* 150cm.
- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian kelima adalah PESAWAT, BUS, MOTOR.

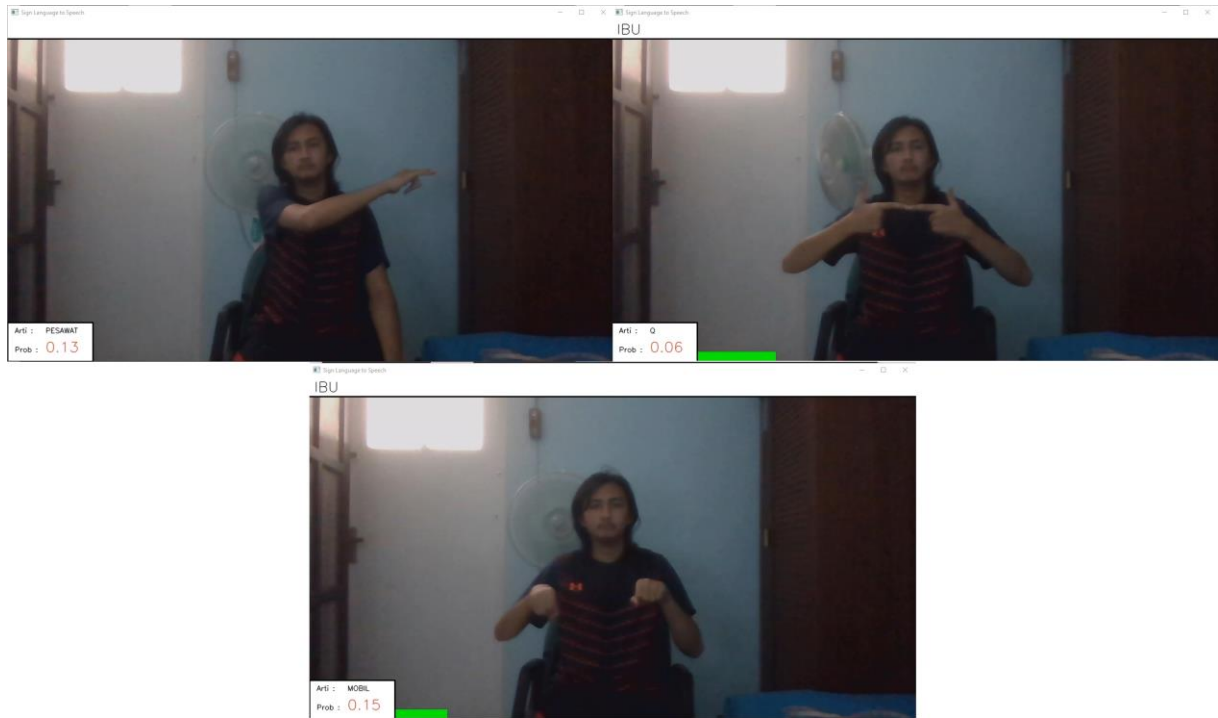


Gambar 4.24 Pengujian 5 “PESAWAT, BUS, MOTOR”

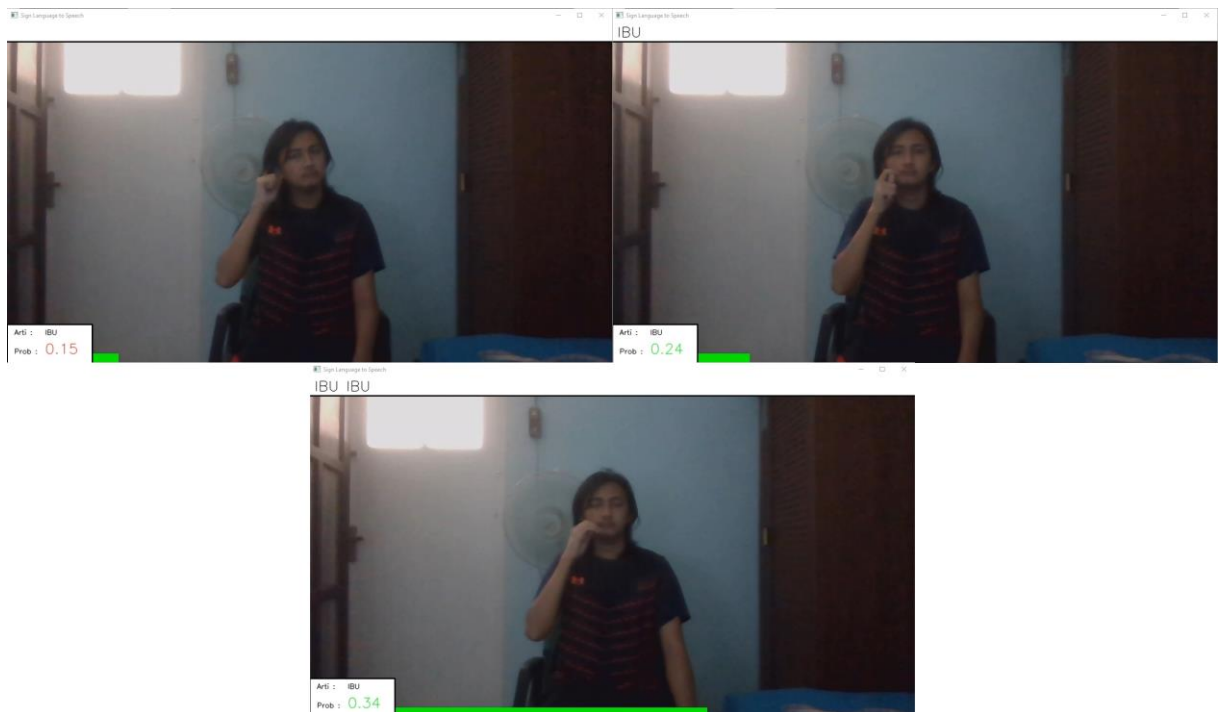
Environment pengujian keenam adalah sebagai berikut:

- Kondisi pencahayaan ruangan redup (diterangi sinar matahari pada siang hari melalui celah di pintu masuk ruangan).
- Jarak pengguna dengan *webcam* 150cm.
- Posisi *webcam* dengan pengguna tegak lurus dan pengguna berada di tengah *frame*.

Bahasa isyarat yang akan dilakukan pengguna pada pengujian keenam adalah PESAWAT, BUS, MOTOR, IBU, TANTE, OM.



Gambar 4.25 Pengujian 6 “PESAWAT, BUS, MOTOR”



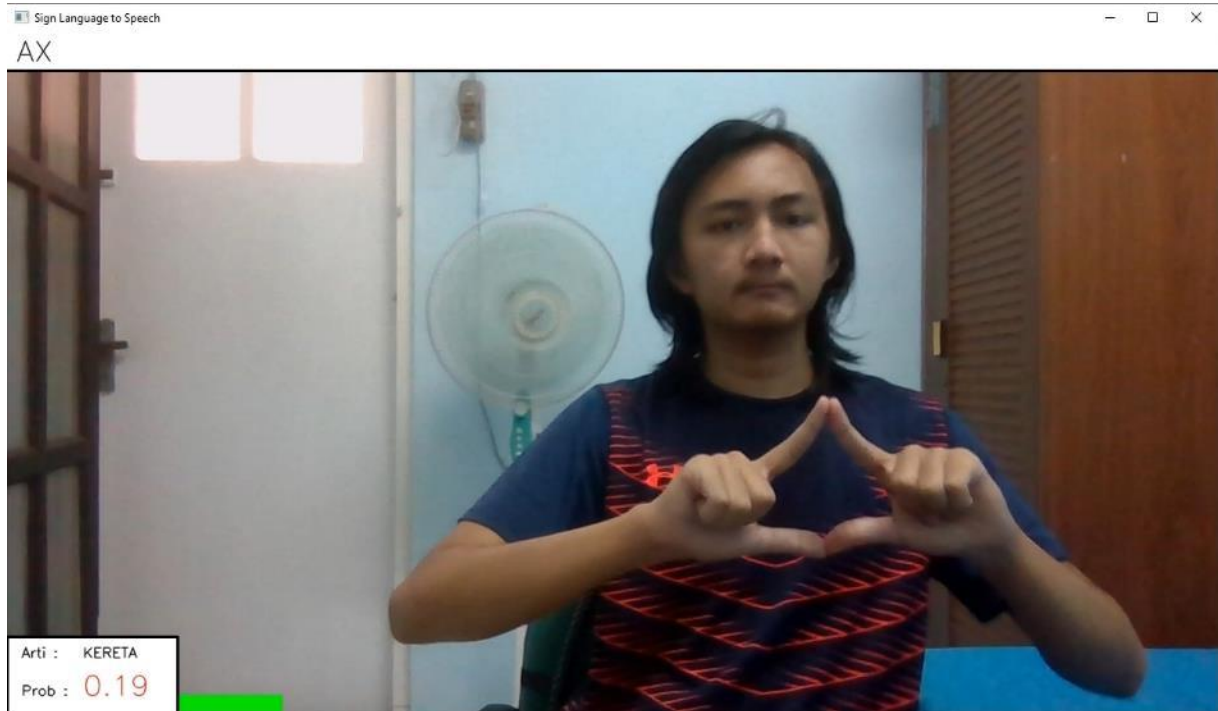
Gambar 4.26 Pengujian 6 “IBU, TANTE, OM”

Environment pengujian ketujuh adalah sebagai berikut:

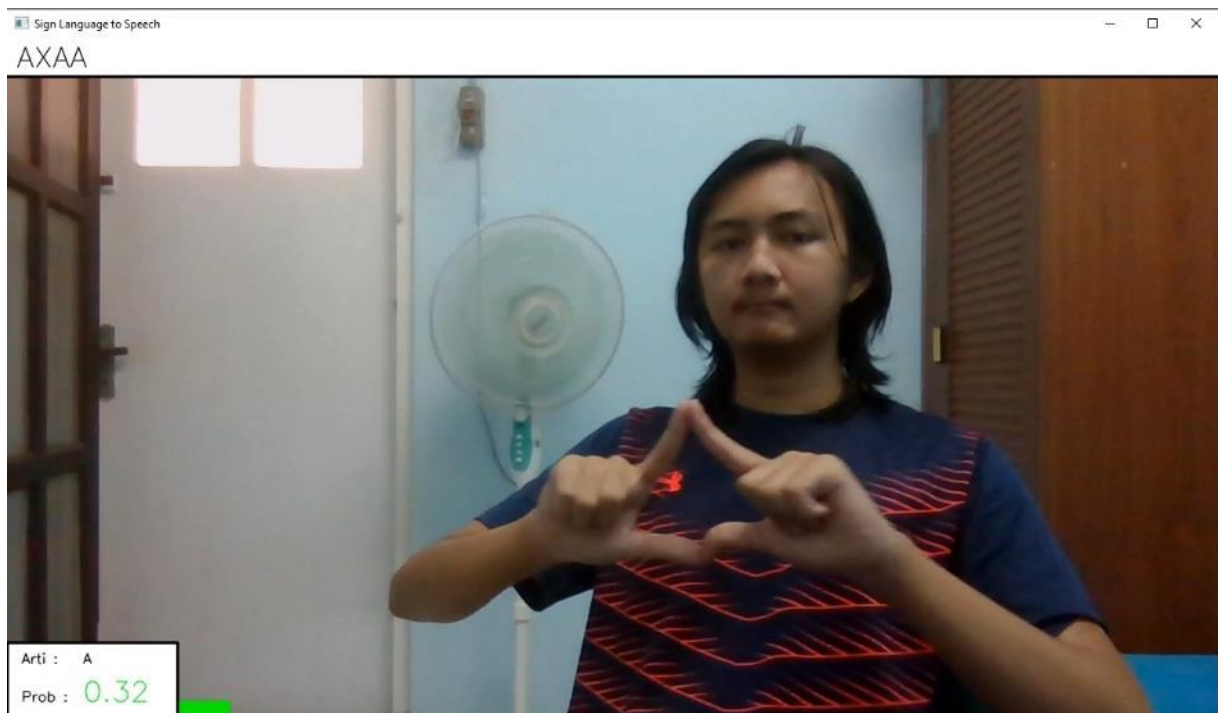
- Kondisi pencahayaan ruangan ruangan terang (diterangi oleh lampu TL LED 36W 220V – 50Hz berposisi diatas langit – langit.).
- Jarak pengguna dengan *webcam* kurang lebih 75cm.

- Posisi *webcam* dengan pengguna tegak lurus dan pengguna tidak berada di tengah *frame*.

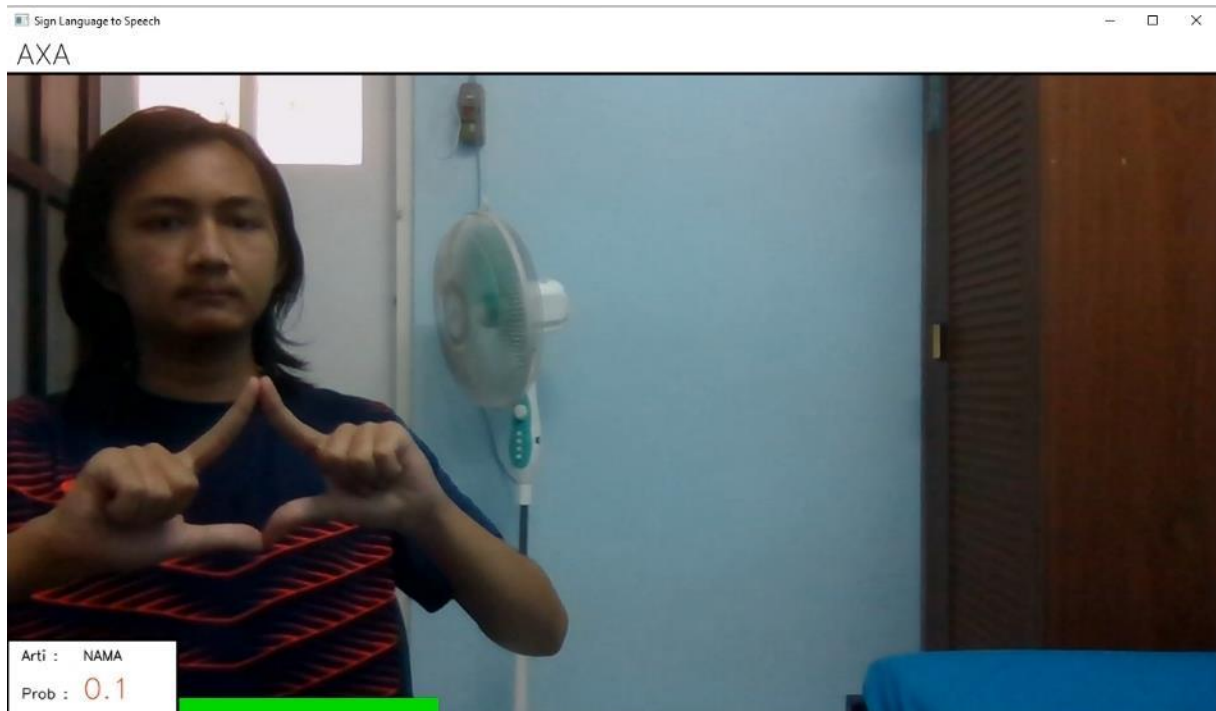
Dalam pengujian ketujuh ini, pengguna akan melakukan bahasa isyarat huruf “A”.



Gambar 4.27 Pengujian 7 Pengguna Berada di Kanan dan Aplikasi gagal Memprediksi



Gambar 4.28 Pengujian 7 Pengguna berada di Kanan dan Aplikasi Berhasil Memprediksi



Gambar 4.29 Pengujian 7 Pengguna Berada di Kiri dan Aplikasi Gagal Memprediksi

Dari hasil pengujian yang dilakukan pada pengguna, dapat ditarik kesimpulan bahwa jarak efektif dari penggunaan aplikasi adalah dalam rentang 75cm hingga 125cm. Hasil pengujian dengan jarak 150cm banyak mengalami *false positive* dan gagal deteksi. Kondisi pencahayaan selama pengguna masih terlihat jelas masih dapat dideteksi oleh *model*. Nilai probabilitas semakin jauh jarak pengguna dari kamera relatif semakin menurun. Data hasil pengujian dapat dilihat pada Tabel 4.7.

Tabel 4.7 Hasil Pengujian Aplikasi pada Pengguna

No.	Isyarat yang dilakukan	Kondisi Pencahayaan	Jarak Pengguna (dalam cm)	Class hasil Prediksi	Probabilitas
1	PESAWAT	Terang	75	PESAWAT	0.4
2	BUS	Terang	75	BUS	0.31
3	MOTOR	Terang	75	MOTOR	0.55
4	IBU	Terang	75	IBU	0.33
5	TANTE	Terang	75	TANTE	0.25
6	OM	Terang	75	OM	0.38
7	PESAWAT	Redup	75	PESAWAT	0.45
8	BUS	Redup	75	BUS	0.46
9	MOTOR	Redup	75	MOTOR	0.4
10	IBU	Redup	75	IBU	0.3
11	TANTE	Redup	75	TANTE	0.22
12	OM	Redup	75	OM	0.24
13	PESAWAT	Terang	125	PESAWAT	0.27
14	BUS	Terang	125	BUS	0.13
15	MOTOR	Terang	125	MOTOR	0.15
16	IBU	Terang	125	IBU	0.22

No.	Isyarat yang dilakukan	Kondisi Pencahayaan	Jarak Pengguna (dalam cm)	Class hasil Prediksi	Probabilitas
17	TANTE	Terang	125	TANTE	0.27
18	OM	Terang	125	OM	0.21
19	PESAWAT	Redup	125	PESAWAT	0.3
20	BUS	Redup	125	BUS	0.07
21	MOTOR	Redup	125	MOTOR	0.1
22	IBU	Redup	125	IBU	0.32
23	TANTE	Redup	125	TANTE	0.23
24	OM	Redup	125	OM	0.22
25	PESAWAT	Terang	150	PESAWAT	0.17
26	BUS	Terang	150	DI	0.1
27	MOTOR	Terang	150	A	0.12
28	PESAWAT	Redup	150	PESAWAT	0.13
29	BUS	Redup	150	Q	0.06
30	MOTOR	Redup	150	MOBIL	0.15
31	IBU	Redup	150	IBU	0.15
32	TANTE	Redup	150	IBU	0.24
33	OM	Redup	150	IBU	0.34

4.2.4 Fitur *Text-to-Speech*

Untuk demonstrasi fitur *text-to-speech* dapat dilihat pada link berikut (link dibuat pada tanggal penulisan laporan ini):

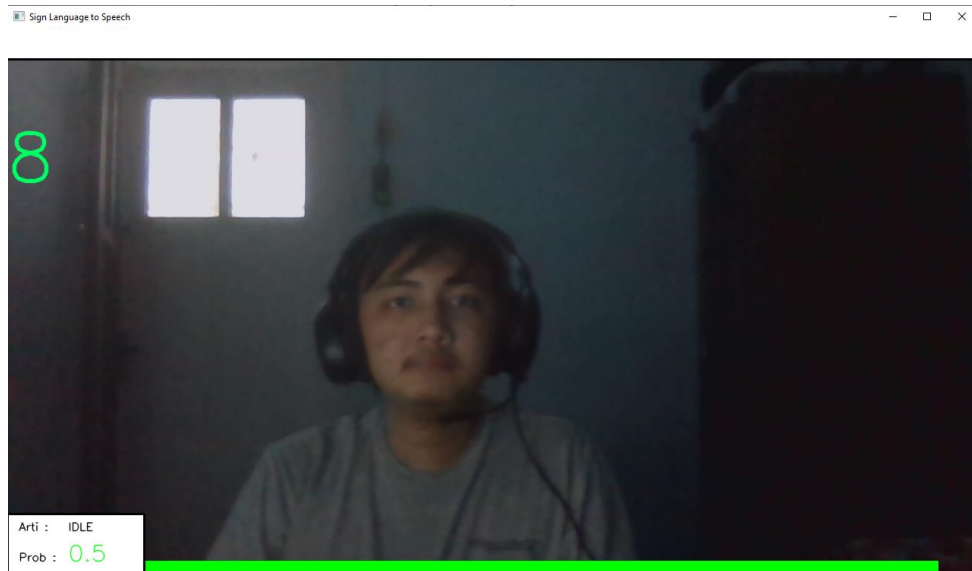
<https://drive.google.com/drive/folders/158BPZUYDoqds6QkuAdjmwTLx7pbpUD4C?usp=sharing>

4.3 Analisis

Analisis dari keseluruhan hasil dan pengujian terhadap *model* dan aplikasi yang diimplementasikan pada penelitian ini.

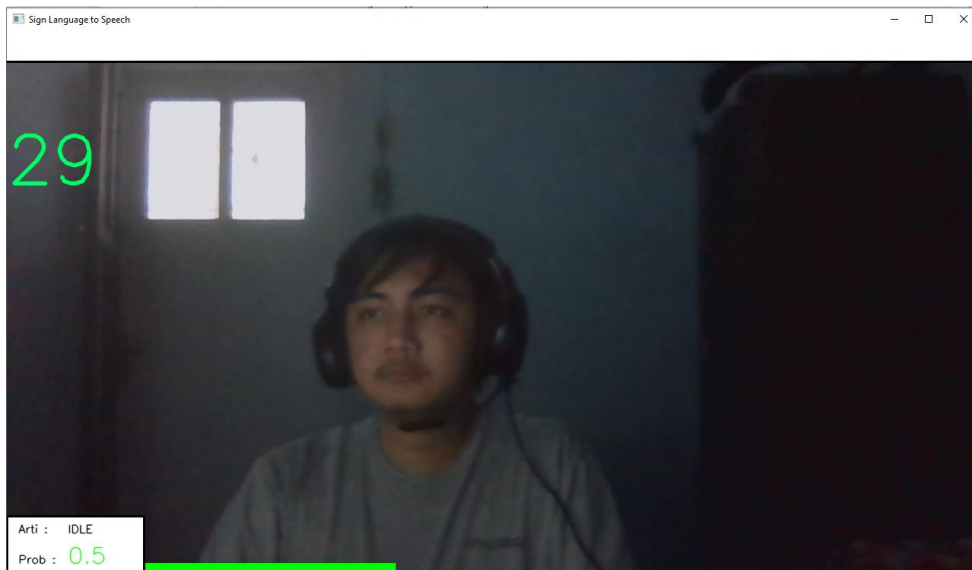
4.3.1 Analisis Performa Aplikasi

Frame rate aplikasi pada saat penggunaan dengan spesifikasi *hardware* yang sudah disebutkan pada Bab 3.2 didapatkan angka sebesar 7-12 *frame per second*. FPS rendah ini disebabkan oleh proses ekstraksi *landmark* yang dilakukan oleh MediaPipe. Terlihat pada Gambar 4.30 ditunjukkan dengan angka 8 berwarna hijau indikator FPS.



Gambar 4.30 *Frame per second* menggunakan MediaPipe

Jika proses ekstraksi landmark MediaPipe tidak dilakukan, menjalankan aplikasi dengan value *landmark* 0, maka didapatkan rerata 30 *frame per second*. Pengujian ini dilakukan dengan cara hanya menonaktifkan panggilan fungsi MediaPipe untuk mengekstraksi *landmark*. Terlihat pada Gambar 4.31 ditunjukkan dengan angka 29 berwarna hijau indikator FPS.



Gambar 4.31 *Frame per second* tanpa MediaPipe

4.3.2 Analisis Model

Setelah dilakukan training model ditemukan bahwa akurasi model tertinggi didapatkan ketika menggunakan algoritma Random Forest dengan skor akurasi 99.7%, akurasi tertinggi selanjutnya adalah algoritma Support Vector Machine dengan skor akurasi 98.7%. Nilai skor akurasi berbeda dengan ketika melakukan *hyperparameter tuning*, hal ini kemungkinan besar dikarenakan dalam *hyperparameter tuning* menggunakan *cross-validation* sedangkan pada saat

training model hanya menggunakan satu kali *train_test_split* sehingga akurasi *hyperparameter tuning* lebih rendah karena skor tersebut adalah rerata dari hasil *cross-validation*.

Dari hasil pengujian aplikasi pada pengguna ditemukan bahwa model saat melakukan prediksi sensitif terhadap posisi tubuh pengguna, hal ini dikarenakan model melakukan prediksi terhadap semua titik *landmark* pengguna (pose dan kedua tangan) sehingga pengguna saat hendak menggunakan aplikasi harus berada di tengah *frame webcam*. Dikarenakan BISINDO menggunakan kedua tangan untuk berisyarat, model tidak menganggap sama tangan kiri dan kanan sehingga pada isyarat yang memang menggunakan tangan kanan tidak dapat dilakukan dengan tangan kiri dan sebaliknya.

Tingkat akurasi model cukup tinggi dengan nilai probabilitas yang relatif rendah untuk bahasa isyarat yang memiliki kemiripan tinggi dengan bahasa isyarat yang lain seperti kata “aku, kita, kamu” dimana perbedaannya sangat minim, untuk bahasa isyarat yang cukup berbeda memiliki tingkat probabilitas yang tinggi seperti kata “semangat”.

Model dapat memprediksi cukup akurat dari jarak 75cm hingga jarak 125cm dengan kondisi pencahayaan ruangan yang cukup agar pengguna dapat terlihat. Bahasa isyarat yang memiliki gerakan di sumbu mendekati *webcam* juga mengalami kendala untuk diprediksi seperti kata “bus”, hal ini terjadi karena MediaPipe saat proses ekstraksi *landmarks* pada tahap pembuatan *dataset* belum bisa memproyeksikan nilai sumbu z dengan baik. Untuk jarak 150cm *model* tidak dapat memprediksi bahasa isyarat dengan baik dan banyak terjadi false positive seperti pada kata “ibu”, “tante”, dan “om”. Pada jarak 125cm keatas pengguna juga akan kesulitan untuk menggunakan fitur *delete* karena jarak *webcam* yang terlalu jauh.

4.3.3 Analisis Aplikasi

Setelah dilakukan pengujian aplikasi didapatkan performa *frame per second* aplikasi adalah 7-12 fps sehingga untuk melakukan 1 deteksi bahasa isyarat diperlukan kurang lebih 2 detik karena 1 deteksi menggunakan 20 *frame*. Hal ini mempengaruhi kenyamanan penggunaan aplikasi karena pengguna tidak dapat melakukan isyarat dengan cepat dan harus menyamakan kecepatan gerakan dengan kemampuan kinerja *hardware*. Pada aplikasi terdapat *display progress bar* yang menandakan jumlah *frame* yang sudah ditangkap agar pengguna dapat mengetahui kapan awal *frame* baru untuk melakukan isyarat.

Faktor yang mempengaruhi hasil akurasi rendah disebabkan oleh gerakan pengguna dengan urutan *frame* bisa tidak sinkron, *dataset* yang digunakan untuk melakukan *training model* semuanya memiliki gerakan yang tepat di setiap *frame*. Faktor lain yang dapat mempengaruhi akurasi rendah atau kegagalan prediksi adalah *landmarks* gagal terdeteksi di salah satu *frame* di saat pengguna melakukan isyarat, hal ini juga mempengaruhi tingkat probabilitas prediksi. Hal ini dipengaruhi kemampuan komputasi *hardware* pengguna dan gerakan pengguna yang terlalu cepat untuk dideteksi. Kondisi pencahayaan ruangan juga mempengaruhi proses ekstraksi *landmark* dimana jika pencahayaan redup, MediaPipe akan mengalami kesulitan untuk mendeteksi adanya manusia atau bagian tubuh pada gambar dan mengakibatkan *landmark* yang dideteksi tidak akurat. Makadari itu ditambahkan fitur *delete* untuk menghapus deteksi yang menurut pengguna adalah salah atau *false positive*.

Dalam penyusunan kalimat terdapat kendala saat pengguna melakukan isyarat abjad (mengeja) untuk 2 kata yang berbeda tanpa ada bahasa isyarat kata (bukan ejaan) ditengahnya, aplikasi tidak mampu mengidentifikasi ketika ejaan sudah membentuk kata dan karena itu aplikasi memberi spasi ‘ ‘ diantara kedua kata tersebut. Hal ini tidak berpengaruh signifikan pada *text-to-speech*, hanya berpengaruh ke *display* kalimat yang dilihat oleh pengguna.

Fitur *text-to-speech* dan fitur *delete* berfungsi dengan baik saat dilakukan pengujian. Fitur *text-to-speech* bersifat *blocking* ketika narator masih membacakan kalimat aplikasi tidak dapat melakukan deteksi bahasa isyarat sampai narator selesai.

BAB V

KESIMPULAN DAN SARAN

5.1 KESIMPULAN

- Aplikasi dapat mendeteksi dan mengartikan bahasa isyarat BISINDO dengan cukup baik dalam rentan jarak 75cm sampai 125cm, hal ini dibuktikan dengan pengujian yang dilakukan terhadap pengguna. Bahasa isyarat dapat dideteksi secara tepat meskipun tingkat probabilitas prediksi relatif rendah (dibawah 0.4).
- Tingkat akurasi prediksi *model* yang digunakan (dengan metode Support Vector Machine) dengan menggunakan dataset penelitian menghasilkan akurasi sebesar 98%. Dengan melakukan uji langsung terhadap pengguna, akurasi *model* sebesar 78%. Hasil akurasi menurun dikarenakan pada pengujian, banyak terjadi gagal prediksi pada jarak 150cm.
- Aplikasi berhasil menghasilkan *output* berupa suara (*audio*) dari bahasa isyarat BISINDO yang dilakukan oleh pengguna menggunakan sistem *text-to-speech*. *Text-to-speech* dapat dilakukan tanpa harus terkoneksi dengan jaringan internet (bekerja secara *offline*).

5.2 SARAN

- Aplikasi dapat dikembangkan lebih lanjut untuk melakukan deteksi bahasa isyarat lainnya seperti SIBI yang juga digunakan di Indonesia.
- Aplikasi dapat dikembangkan lebih lanjut dengan melakukan integrasi pada kamera atau *webcam* eksternal yang dapat melakukan *tracking* pengguna agar pengguna bisa menggunakan aplikasi tanpa harus diam di satu tempat.
- Narator *text-to-speech* dapat dikembangkan lebih lanjut agar dapat menggunakan profil suara yang lebih bervariasi.

DAFTAR PUSTAKA

- Christensson, P. (2008, March 6). *Webcam Definition*. Dipetik March 17, 2022, dari TechTerms.com: <https://techterms.com/definition/webcam>
- Gomase, K., Dhanawade, A., Gurav, P., & Lokare, S. (2022). Sign Language Recognition using Mediapipe. *International Research Journal of Engineering and Technology (IRJET)*, 09(01), 744-746. Diambil kembali dari <https://www.irjet.net/archives/V9/i1/IRJET-V9I1133.pdf>
- Google LLC. (2020). *Hands - mediapipe*. Dipetik June 26, 2022, dari MediaPipe: https://google.github.io/mediapipe/images/mobile/hand_landmarks.png
- Google LLC. (2020). *Home - mediapipe*. Dipetik March 16, 2022, dari MediaPipe: <https://google.github.io/mediapipe/>
- Google LLC. (2020). *Pose - mediapipe*. Dipetik June 26, 2022, dari MediaPipe: <https://google.github.io/mediapipe/solutions/pose.html>
- Halder, A., & Toyade, A. (2021). Real-time Vernacular Sign Language Recognition using MediaPipe and Machine Learning. *International Journal of Research Publication and Reviews*(5), 9-17. Diambil kembali dari <https://ijrpr.com/uploads/V2ISSUE5/IJRPR462.pdf>
- IBM. (1986, March). *What is the k-nearest neighbors algorithm?* | IBM. Dipetik July 24, 2022, dari IBM: <https://www.ibm.com/id-en/topics/knn>
- Intel Corporation. (2022). *About - OpenCV*. Dipetik March 17, 2022, dari OpenCV: <https://opencv.org/about/>
- JavaTpoint. (2011). *Machine Learning Random Forest Algorithm - Javatpoint*. Dipetik April 7, 2022, dari javaTpoint: <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
- JavaTpoint. (2011). *Support Vector Machine (SVM) Algorithm - JavaTpoint*. Dipetik July 24, 2022, dari JavatPoint: <https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm>
- Kementerian Sosial RI. (2018). *Sistem Informasi Penyandang Disabilitas - Kementerian Sosial RI*, 2. Dipetik March 15, 2022, dari Sistem Informasi Management Penyandang Disabilitas: <https://simpd.kemensos.go.id/>
- Klobility. (2019, April 5). *Klobility - BISINDO dan SIBI: Apa Bedanya?* Dipetik 6 26, 2022, dari Klobility: <https://www.klobility.id/post/perbedaan-bisindo-dan-sibi>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., . . . Édouard, D. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- PUSBISINDO. (2022). *PUSBISINDO | Pusat Bahasa Isyarat Indonesia*. Dipetik July 29, 2022, dari PUSBISINDO: <https://pusbisindo.org/>
- Tempo Media Group. (2021, December 4). *Sama-sama Bahasa Isyarat, Apa Perbedaan SIBI dan BISINDO?* (Nurhadi, Editor) Retrieved March 15, 2022, from TEMPO.CO: <https://nasional.tempo.co/read/1535664/sama-sama-bahasa-isyarat-apa-perbedaan-sibi-dan-bisindo>
- WETA. (2000, March 13). *Text-to-Speech Technology: What It Is and How It Works | Reading Rockets*. Dipetik June 26, 2022, dari Reading Rockets: <https://www.readingrockets.org/article/text-speech-technology-what-it-and-how-it-works>
- Widinarsih, D. (2019, October 2). Penyandang Disabilitas Di Indonesia: Perkembangan Istilah Dan Definisi. *Jurnal Ilmu Kesejahteraan Sosial*, 20(2), 127-142.

- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020, June 18). MediaPipe Hands: On-device Real-time Hand Tracking. doi:<https://doi.org/10.48550/arXiv.2006.10214>
- Zhang, Y. (2012). Support Vector Machine Classification Algorithm and Its Application. *Information Computing and Applications*, 179-186. doi:https://doi.org/10.1007/978-3-642-34041-3_27

BIODATA PENULIS



Muhammad Zulfikar Fauzi dilahirkan di Surabaya, 29 Agustus 2001. Penulis telah menempuh pendidikan formal tingkat SD di SDI Sabilillah Sidoarjo lulus pada tahun 2013, SMP di SMP Muhammadiyah 1 Sidoarjo lulus pada tahun 2016, SMA di SMAN 3 Sidoarjo lulus pada tahun 2018 Melalui jalur akselerasi dan sekarang sedang menjadi mahasiswa melalui jalur SNMPTN di Institut Teknologi Sepuluh Nopember program studi Teknologi Informasi Fakultas FTEIC mulai tahun 2018 dengan NRP 05311840000012.

Selama masa perkuliahan penulis aktif dalam mengikuti kegiatan organisasi mahasiswa seperti himpunan, dimana penulis menjabat sebagai sekretaris departemen di periode pertama (2019 – 2020) dan staff ahli departemen di periode selanjutnya (2020 – 2021), dan kegiatan UKM Kendo ITS yang dimana penulis juga pernah menjabat sebagai salah satu staff pengurus (2019 – 2020).

Penulis juga cukup aktif dalam mengikuti kegiatan seminar maupun webinar yang bertemakan teknologi informasi dan kerap mengikuti perlombaan di bidang tersebut juga, salah satunya adalah gemastik. Penulis juga ikut serta dalam ajang ARA HMIT ITS, dimana pada ARA HMIT ITS pertama penulis menjadi kakak pendamping dalam melakukan pelatihan bertemakan *Internet of Things*.